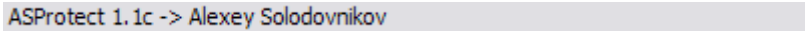
Programa:	Cartoonist 1.1
Protección:	ASProtect 1.1c + nags + inserta watermark en las magenes creadas
Objetivos:	Desempakado de ASProtect 1.1c (1ª parte)
URL descarga:	http://www.helpandmanual.com
Dificultad:	La IAT casi agota la paciencia del cracker ;-)
Herramientas:	OllyDbg v1.10 + PEiD 0.94 + ImpRec 1.6
Cracker:	SyXe'05 ... [hAcK-c0d3d] ... [CracksLatinoS] Fecha: 04-01-06

Hola a todo CracksLatinoS!!

Una vez más aquí reunidos para estudiar juntos el comportamiento de una protección, ya os adelanto que el tute puede resultar engañoso por la versión del packer pero a la vez reconfortante, ya lo veremos más adelante, por lo pronto decir que es de las protecciones más antiguas de ASProtect, esto hace pensar ¡qué facil! bua esto lo hago yo en dos minutos, pues bien adelante, tomad asiento no es gran cosa eso sí pero tiene una manera magistral diría yo de redirigir la IAT, bueno eso lo veremos más adelante también, de momento decir que la víctima será el Cartoonist 1.1 un programita del que se comentó algo hace poco en la lista, creo recordar que fue "txino crk" quien además preguntaba por manuales del ASProtect 1.1c, y como parece que no encontró recurrió a la lista con este programa.

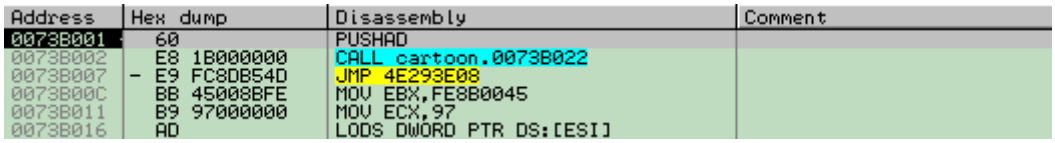
Dicho esto pasaremos al ataque, lo primero es lo primero dijo un sabio, nosotros ahora sabemos que protección tiene pero cuando uno instala un progama así de primeras lógicamente no tiene ni idea, lo que haremos será abrirnos el PEiD 0.94 que tanto nos ha ayudado y mirar a ver qué tiene:



Ahora sí lo tenemos ASProtect 1.1c, como digo una de las versiones más antiguas que tiene, no por esto debemos menospreciarla, ni es tan mala ni es tan buena, lo mejor de todo creo que será verla y que cada uno juzgue por sí mismo ok?

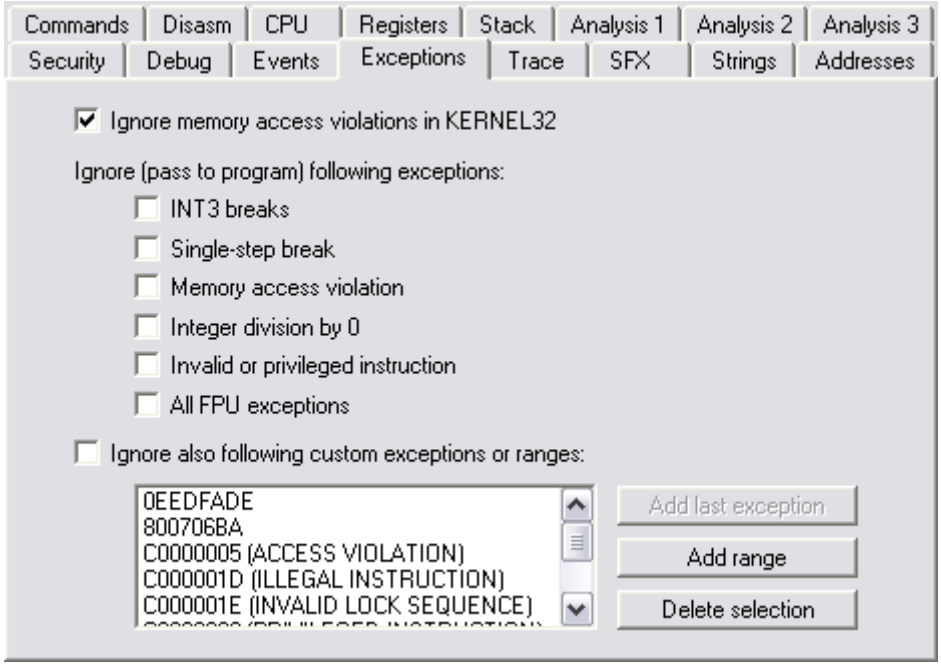
Cerramos PEiD y nos abrimos un Olly, he de decir que tengo veinticinco mil plugins puestos así que enumerarlos todos sería de locos, hace poco bajé la versión full que los traía consigo "de serie" al que le he añadido el HideOD que trae todavía más opciones, si cabe, para ocultar el debugger (el Olly vaya) esto no hace mucha falta porque la única detección que hace ASProtect es mediante la api IsDebuggerPresent así que no hay que preocuparse demasiado por el tema.

Visto que antidebug apenas tiene sólo nos queda comenzar ;-) En el olly que tenemos abierto cargamos el programa, y quedamos parados en el EntryPoint del mismo:

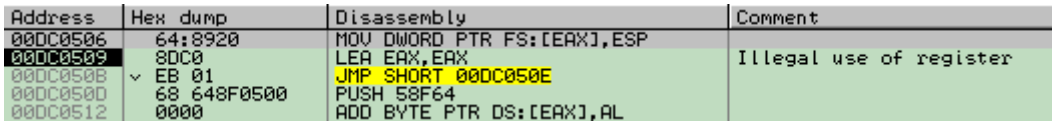


Address	Hex dump	Disassembly	Comment
0073B001	60	PUSHAD	
0073B002	E8 1B000000	CALL cartoon.0073B022	
0073B007	- E9 FC8DB54D	JMP 4E293E08	
0073B00C	BB 45008BFE	MOV EBX,FE8B0045	
0073B011	B9 97000000	MOV ECX,97	
0073B016	AD	LODS DWORD PTR DS:[ESI]	

Este es el típico EP de ASProtect, lleva nosecuantas versiones sin cambiar ni un ápice, jeje.. pero ni que decir tiene que el método del PUSHAD-POPAD no servirá, lo primero que haremos como siempre será descubrir el OEP, el método que aquí utilizaremos es el de Ricardo (método de las excepciones) esta es una de sus genialidades, la verdad que es prácticamente infalible, en pocos casos falla (en casi ninguno) valiéndonos de esa técnica atacaremos, lo que tenemos que hacer es primero de todo destildar todas las excepciones para que Olly se vaya deteniendo en cada una de ellas, para hacer esto nos vamos a "Options->Debugging options" y allí a la pestaña "Exceptions" otra manera más cómoda de hacerlo sería simplemente pulsando Alt+O, así aparecerá el cuadro de configuración de Olly donde podemos modificar infinidad de cosas, pues allí nos vamos al cuadro de Exceptions y lo dejamos de esta manera:

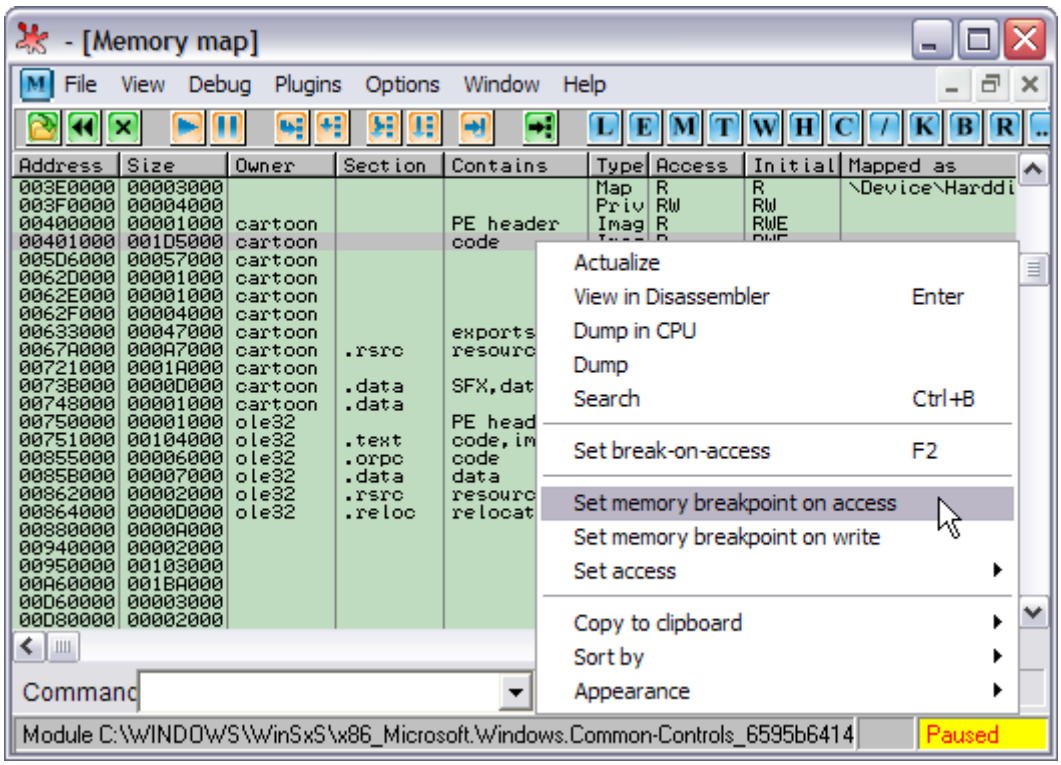


Así Olly irá parando cada vez que ocurra una excepción, jeje.. lo siguiente es dejar correr el programa e irán saltando las excepciones que iremos superando con Shift+F9, llegará un momento en que no habrá más y el programa cargará, anotamos la última que ocurrió, en mi caso es esta:



Olly ya sabe que va a suceder una excepción incluso antes pues mirad que pone al lado el comentario "Illegal use of register" si miramos la barra de estado cuando estemos en plena excepción veremos algo así:

O sea que seprodujo una instrucción ilegal, claro hubo un uso "ilegal" del registro EAX como bien nos decía Olly, entonces conocida la última excepción reiniciamos el proceso con Ctrl+F2 y volvemos a llegar hasta ella pero ahora en lugar de superarla como hicimos antes nos vamos a la ventana de memoria con Clt+M y ponemos un BPM en la primera sección después de la cabecera (PE header):



Una vez puesto entonces sí superamos la excepción con Shift+F9 y estamos en el OEP:

```
- [CPU - main thread, module cartoon]
File View Debug Plugins Options Window Help

Address   Hex dump   Disassembly
00401000  EB 10      JMP SHORT cartoon.00401012
00401002  66:623A    BOUND DI,DWORD PTR DS:[EDX]
00401005  43         INC EBX
00401006  2B2B       SUB EBP,DWORD PTR DS:[EBX]
00401008  48         DEC EAX
00401009  4F         DEC EDI
0040100A  4F         DEC EDI
0040100B  4B         DEC EBX
0040100C  90         NOP
0040100D  E9 68665D00  JMP 009D767A
00401012  A1 5B665D00  MOV EAX,DWORD PTR DS:[5D665B]
00401017  C1E0 02    SHL EAX,2
0040101A  A3 5F665D00  MOV DWORD PTR DS:[5D665F],EAX

Registers (MMX)
EDX 7FFE0304
EBX 7FFDF000
ESP 0012FFC4
EBP 0012FFF0
ESI BF878225
EDI BF87B2F2
EIP 00401000 cartoon.00

00401012=cartoon.00401012

Address   Hex dump
0073B000  90 60 E8 1B 00 00 00 E
0073B010  FE B9 97 00 00 00 AD 3
0073B020  EB 0A 5D 45 55 81 ED 5
0073B030  00 E8 01 00 00 00 90 5
0073B040  B6 45 00 03 DD 2B 9D B

0012FFC4  77E614C7  RETURN to kernel32.77E614C7
0012FFC8  BF87B2F2
0012FFCC  BF878225
0012FFD0  7FFDF000
0012FFD4  F5991CF0
0012FFD8  0012FFC8

Command

Memory breakpoint when executing [00401000]   Paused
```

Este es nuestro OEP, el .exe además está compilado en ensamblador, al menos el patrón genérico de estos compiladores (TASM, MASM.,) es colocar el OEP en 401000 como vemos en la imagen, antes de que se nos olvide ns anotamos el dato ;-)

OEP: 401000

Ahora que estamos aquí buscaremos la IAT, el método es sencillo, buscamos alguna instrucción tipo "CALL DWORD PTR [xxxxxxxx]" ó "JMP DWORD PTR [xxxxxxxx]" cuyos opcodes son FF15 y FF25 respectivamente, y no son más que una llamada y un salto a la IAT, a mi me gusta más ir entrando en el primer CALL (aquí está en :401022 y buscarlos a mano, precisamente entramos en el CALL citado y mirad que vemos ;-)

```
Address   Hex dump        Disassembly                  Comment
005D4926  - FF25 ECF36200 JMP DWORD PTR DS:[62F3EC]
005D492C  - FF25 F0F36200 JMP DWORD PTR DS:[62F3F0]
005D4932  - FF25 F4F36200 JMP DWORD PTR DS:[62F3F4]
005D4938  - FF25 F8F36200 JMP DWORD PTR DS:[62F3F8]
005D493E  - FF25 FCF36200 JMP DWORD PTR DS:[62F3FC]
005D4944  - FF25 00F46200 JMP DWORD PTR DS:[62F400]
005D494A  - FF25 04F46200 JMP DWORD PTR DS:[62F404]
005D4950  - FF25 08F46200 JMP DWORD PTR DS:[62F408]
005D4956  - FF25 0CF46200 JMP DWORD PTR DS:[62F40C]
005D495C  - FF25 10F46200 JMP DWORD PTR DS:[62F410]
005D4962  - FF25 14F46200 JMP DWORD PTR DS:[62F414]
005D4968  - FF25 18F46200 JMP DWORD PTR DS:[62F418]
005D496E  - FF25 1CF46200 JMP DWORD PTR DS:[62F41C]
005D4974  - FF25 20F46200 JMP DWORD PTR DS:[62F420]
005D497A  - FF25 24F46200 JMP DWORD PTR DS:[62F424]
005D4980  - FF25 28F46200 JMP DWORD PTR DS:[62F428]
005D4986  - FF25 2CF46200 JMP DWORD PTR DS:[62F42C]
005D498C  - FF25 30F46200 JMP DWORD PTR DS:[62F430]
005D4992  - FF25 34F46200 JMP DWORD PTR DS:[62F434]
005D4998  - FF25 38F46200 JMP DWORD PTR DS:[62F438]
005D499E  - FF25 3CF46200 JMP DWORD PTR DS:[62F43C]
005D49A4  - FF25 40F46200 JMP DWORD PTR DS:[62F440]
005D49AA  - FF25 44F46200 JMP DWORD PTR DS:[62F444]
```

Esta es la tabla de saltos (JMP_Table) que apuntan a la IAT, en este caso está bastante cascada y no vemos el nombre de ninguna API al lado, cosa que indica que la tabla está totalmente o casi totalmente redirigida, cosas de ASProtect.. de momento la delimitamos, nos ponemos sobre uno cualquiera de los JMP y pulsamos con el botón derecho, en el menú que aparece elegimos "Follow in dump->Memory Address" y vemos esto:

```
Address   Hex dump                                         ASCII
0062F3EC  3C 49 DD 00 48 49 DD 00 54 49 DD 00 08 C2 DB 00  <I¦.HI¦.TI¦.■┬■.
0062F3FC  64 49 DD 00 74 49 DD 00 94 49 DD 00 A4 49 DD 00  dI¦.tI¦.öI¦.ñI¦.
0062F40C  B4 49 DD 00 C0 49 DD 00 CC 49 DD 00 D8 49 DD 00  ┤I¦.└I¦.╠I¦.╪I¦.
0062F41C  E8 49 DD 00 F8 49 DD 00 08 4A DD 00 14 4A DD 00  ╘I¦.°I¦.■J¦.¶J¦.
0062F42C  20 4A DD 00 34 4A DD 00 44 4A DD 00 50 4A DD 00   J¦.4J¦.DJ¦.PJ¦.
```

Allí es donde apuntaba el salto que seguí, ahora subimos un poco con los cursores o con la ruedecilla del ratón hasta encontrar el inicio de la tabla, hay que tener cuidado que hay lugares en los que parece que termina pero no es así, si seguimos mirando para arriba encontramos más entradas allí, el inicio es este:

Address	Hex dump	ASCII
0062F0F4	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0062F104	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0062F114	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0062F124	50 10 00 10 F0 10 00 10 00 00 00 00 00 00 00 00	P►.►-►.►.....
0062F134	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0062F144	00 00 00 00 00 00 00 00 00 00 00 00 D8 17 DA 77	.....ï‡ rw
0062F154	D6 27 DA 77 E2 68 DB 77 6D 69 DB 77 9A 22 DA 77	i' rwôh▄wmi▄wü" rw
0062F164	10 24 DA 77 B1 63 DB 77 00 00 00 00 00 00 00 00	►$ rw▒c▄w.....
0062F174	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0062F184	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0062F194	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....

Vemos lo que os decía que la IAT está muy agujereada como se suele decir, entre DLL y DLL hay muchos ceros, de momento nos anotamos este valor también:

inicio_iat: 62F124

Y hacemos lo mismo con el final que encontramos aquí:

Address	Hex dump	ASCII
0062FFD4	D3 12 34 76 DA 14 34 76 62 36 34 76 20 3B 34 76	ë‡4v┌¶4vb64v ;4v
0062FFE4	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0062FFF4	00 00 00 00 C6 82 76 00 17 0E 76 00 7C C3 7A 00	....ãév.♫Av.│├z.
00630004	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00630014	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00630024	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00630034	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00630044	00 00 00 00 56 38 10 77 B1 CC 12 77 67 33 0F 77	....V8►w▒╠‡wg3*w
00630054	7F 32 0F 77 E4 F3 0F 77 22 30 0F 77 BE 2E 0F 77	⌂2*w▒%*w"0*w¥.*w
00630064	DC CC 12 77 74 16 0F 77 4B 17 0F 77 2E F4 0F 77	▄╠‡wt▬*wK‡*w.¶*w
00630074	8F 3D 0F 77 E2 19 0F 77 EC 14 0F 77 B8 AB 0F 77	Å=*w▒↓*w▒¶*w@½*w
00630084	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00630094	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
006300A4	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....

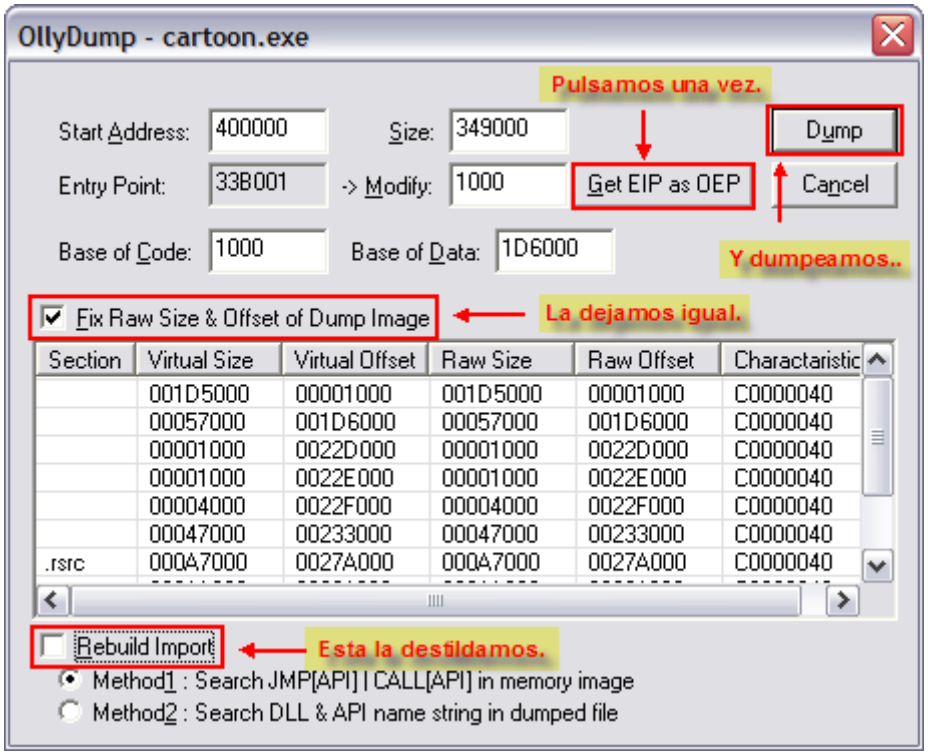
que también anotamos por su puesto:

final_iat: 630084

El largo de la misma es la diferencia entre ambos valores:

Longitud_iat: F64

Vale vamos a por lo siguiente, como no hay Stolen Bytes podemos dumpear ya desde aquí puesto que estamos parados en el OEP del programa, las opciones serán estas:



Las opciones son las que solemos poner siempre (salvo excepciones claro) al final se abrirá un pequeño explorador donde le damos nombre tute.exe por ejemplo, así nos referimos todos al mismo archivo, y aquí comienza lo duro, jeje..

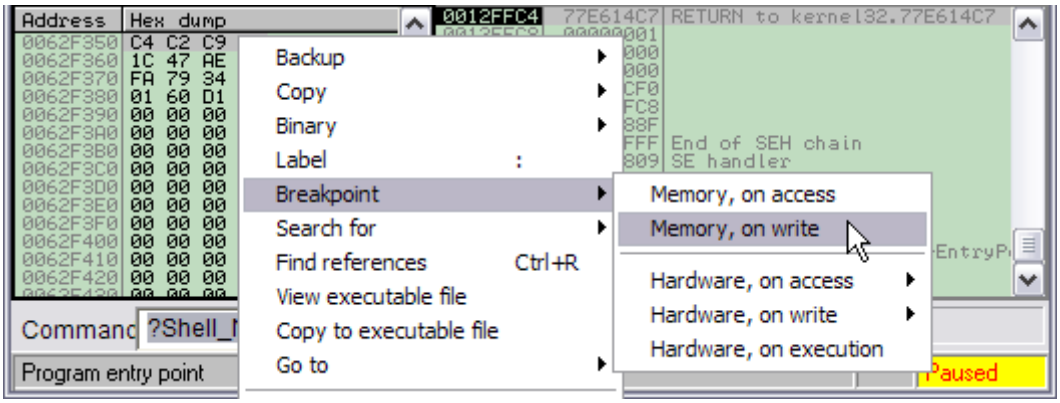
Recuperando las Importaciones...

Es sabido que ASProtect es uno de los packers que más destroza la IAT, pero en esta versión he encontrado que lo hace de una manera un tanto violenta, sigamos con el trabajo y veamos a lo que me refiero.

Como todavía tenemos el progama parado en el OEP nos valdremos de él para recuperar algunos valores que nos harán falta, os explico, en este momento nos interesa conocer la dirección de una de las entradas de la tabla que va a ser redirigida por el packer,y por su puesto su valor final, seamos prácticos y avanzaremos más, este es el primero:

```
Address   Hex dump                                          ASCII
0062F330  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0062F340  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0062F350  34 03 DD 00 D4 46 DD 00 E0 02 DD 00 E0 46 DD 00  4♥¦.ÔF¦.à☻¦.àF¦.
0062F360  F0 46 DD 00 00 47 DD 00 10 47 DD 00 20 47 DD 00  -F¦..G¦.►G¦. G¦.
0062F370  40 47 DD 00 50 47 DD 00 60 47 DD 00 70 47 DD 00  @G¦.PG¦.'G¦.pG¦.
0062F380  7C 47 DD 00 9C 47 DD 00 B4 47 DD 00 C4 47 DD 00  |G¦.£G¦.┤G¦.─G¦.
```

Vemos que la primera entrada redirigida la encontramos en la dirección :62F350 y su valor final es 00DD0334 (se lee al revés recordad) entonces lo que haremos será reiniciar Olly y poner un "BPM On Acces" cuando estemos en el EP del programa para ver donde se escriben estos valores malos:



una vez puesto el BPM vamos pulsando F9 para que el programa corra, cada vez que el programa vaya a escribir algo allí Olly parará, entonces debemos mirar si se escribieron los bytes 00DD0334 (los finales) u otra cosa:

```
Address   Hex dump   Disassembly                      Comment
00DB93D6    8908     MOV DWORD PTR DS:[EAX],ECX
00DB93D8    8958 04  MOV DWORD PTR DS:[EAX+4],EBX
00DB93DB    5A       POP EDX
00DB93DC    5D       POP EBP
00DB93DD    5F       POP EDI
00DB93DE    5E       POP ESI
00DB93DF    5B       POP EBX
00DB93E0    C3       RETN
```

para ir más rápidos podemos tildar las excepciones para que Olly no pare pero sobre todo hay que recordar destildarlas después, en este caso no interesa pus escribió otros datos allí, en concreto BE4170BA, no nos interesa, pulsamos F9 y vuelve a parar, nada no pone lo que buscamos, así un par de veces más hasta caer aquí:

```
Address   Hex dump   Disassembly                      Comment
00DBF0DD    8902     MOV DWORD PTR DS:[EDX],EAX
00DBF0DF    E9 F8000000  JMP 00DBF1DC
00DBF0E4    E8 01000000  CALL 00DBF0EA
00DBF0E9    E9 83C404FF  JMP FFE0B571
00DBF0EE    35 1BF1DB00  XOR EAX,0DBF11B
```

Si miramos en la ventana de registros vemos que EAX sí tiene el valor que buscamos, está claro que está rellenando la tabla en estos momentos:

```
Registers (MMX)
EAX 00DD0334
ECX 00000000
EDX 0062F350 cartoon.0062F350
EBX 0012FE1B ASCII "CloseHandle"
ESP 0012FE0C
EBP 0012FF38
ESI 0012FF7C
EDI 00DD2693
```

Además en EBX está el nombre de una API, jeje.. está claro que estamos cerca, si nos desplazamos hacia arriba con los cursores pero no mucho mirad lo que encontramos ;-)

```
Address   Hex dump     Disassembly                      Comment
00DBF0CB    53         PUSH EBX
00DBF0CC    8B45 0C    MOV EAX,DWORD PTR SS:[EBP+C]
00DBF0CF    40         INC EAX
00DBF0D0    50         PUSH EAX
00DBF0D1    E8 BAFAFFFF CALL 00DBEB90
00DBF0D6    E8 79FEFFFF CALL 00DBEF54      call semi-mágico!
00DBF0DB    8B16       MOV EDX,DWORD PTR DS:[ESI]
00DBF0DD    8902       MOV DWORD PTR DS:[EDX],EAX
00DBF0DF  v E9 F8000000 JMP 00DBF1DC
```

jeje.. es el call semi-mágico que descubrió Juan José en una de sus sesiones locas de cracking xDDDDDD un saludo compañero! vamos a ver porqué es este el CALL buscado y no otro, pongamos un BP en el PUSH EBX que vemos en la imagen anterior:

```
Address   Hex dump     Disassembly                      Comment
00DBF0CB    53         PUSH EBX
00DBF0CC    8B45 0C    MOV EAX,DWORD PTR SS:[EBP+C]
00DBF0CF    40         INC EAX
00DBF0D0    50         PUSH EAX
00DBF0D1    E8 BAFAFFFF CALL 00DBEB90
00DBF0D6    E8 79FEFFFF CALL 00DBEF54
00DBF0DB    8B16       MOV EDX,DWORD PTR DS:[ESI]
```

Damos RUN (pulsando F9) y Olly para en el BP, aquí todavía no vemos nada, bueno sí, el nombre de una api en los registros ECX y EBX pero nada importante, de repente si traceamos un poco con F8 encontramos que tras pasar por el CALL 00DBEB90 nos queda una API en el registro EAX:

```
Address   Hex dump     Disassembly                      Comment
00DBF0D0    50         PUSH EAX
00DBF0D1    E8 BAFAFFFF CALL 00DBEB90                   ;pone una api en EAX..
00DBF0D6    E8 79FEFFFF CALL 00DBEF54
00DBF0DB    8B16       MOV EDX,DWORD PTR DS:[ESI]
```

Si miramos en los registros encontramos la api tratada:

```
Registers (MMX)
EAX 77E54DE5 kernel32.CreateEventA
ECX 0012FDFC
EDX 00000000
EBX 0012FE1B ASCII "CreateEventA"
ESP 0012FE0C
```

CreateEventA parece ser la api que toca en este momento, si ahora superamos el siguiente call con F8 (el semi-mágico) vemos que la api desaparece convirtiéndose en un valor MALO de los de ASProtect:

```
Address   Hex dump     Disassembly                      Comment
00DBF0D0    50         PUSH EAX
00DBF0D1    E8 BAFAFFFF CALL 00DBEB90                   ;pone una api en EAX..
00DBF0D6    E8 79FEFFFF CALL 00DBEF54                   ;nos roba la api!
00DBF0DB    8B16       MOV EDX,DWORD PTR DS:[ESI]       cartoon.0062F358
00DBF0DD    8902       MOV DWORD PTR DS:[EDX],EAX
```

Y si miramos ahora nuevamente los registros vemos esto:

```
Registers (MMX)
EAX 00DD02E0      api mangada ;-(
ECX 00000000
EDX 0012FFE0
EBX 0012FE1B ASCII "CreateEventA"
ESP 0012FE0C
```

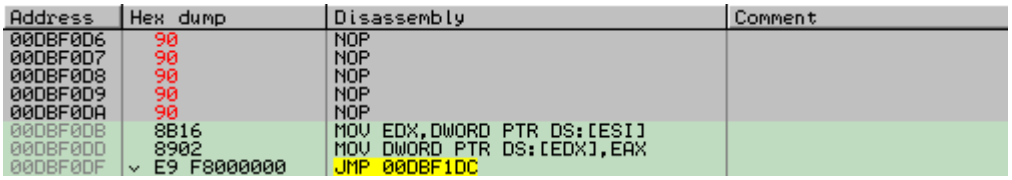
vaya nos robó la api ese CALL! es por esto que lo debemos nopear, como el CRC32 (checksum) que tiene ASProtect no nos deja nopear impúnemente lo que haremos será un nopeo temporal, jeje.. es decir, nopearemos el CALL para sacar partido y una vez tengamos el botín lo dejaremos en su sitio, como los ladrones de guante blanco jeje! para hacer esto necesitamos la dirección del CALL:

call semi_mágico: 0DBF0D6

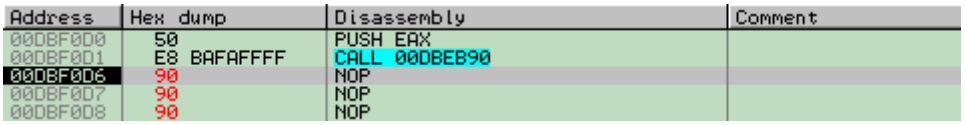
Reiniciamos Olly nuevamente y desde el EP pulsamos F9 para dejar correr al programa, parará en la primera excepción, por eso decía antes que si las tildábamos deberíamos recordar tildarlas ya que ahora las necesitamos, además Olly debe parar en la primera para hacer el "Go" que nos lleve hasta la dirección del CALL a nopear, si no hacemos esto Olly nos dará un error porque estaremos intentando acceder a una zona de memoria que todavía no existe, ASProtect las va creando en tiempo de ejecución. Esperamos a que cree dicha sección y es entonces cuando hacemos el "Go" ok? como en la primera excepción la memoria ya está acesible esta zona volvemos a tildar las excepciones (para que Olly no pare en ellas) y nos vamos hasta allí con el "Go"(puede parecer lioso pero creedme que es súper sencillo esto cuando se hace dos o tres veces seguidas)

Suponiendo que estamos parados en la primera excepción pulsamos Ctrl+G y se abre un pequeño cuadro donde pondremos la dirección a la que queremos ir, (importante poner un cero delante de las letras cuando son el primer carácter sinó Olly dará error):

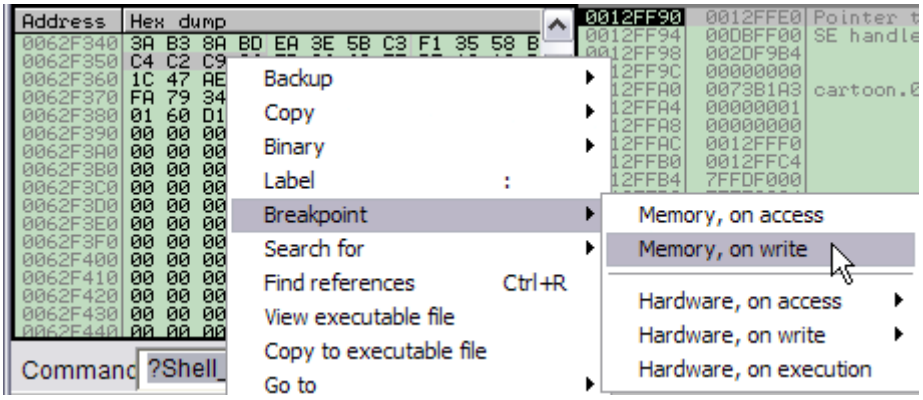
Aceptamos y caemos sobre el CALL semi-mágico, lo nopeamos.. jeje



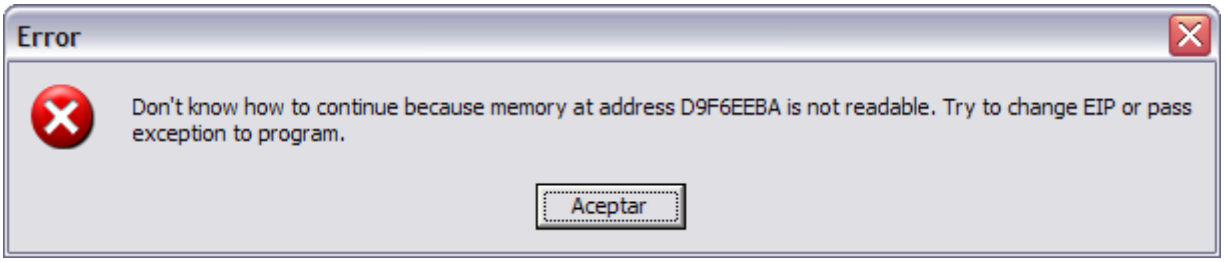
Ahora ponemos un BP en esta dirección para ahorrar faena y tildamos las excepciones para que no pare, pulsamos F9 y estamos allí parados:



Antes de dejar correr el programa nos falta algo importante: digamos ponerle un freno pues si lo dejamos tal cual no correrá debido a las comprobaciones que hace en el código (detecta la modificación), para evitar esto vamos a la última entrada de la tabla y le ponemos un BPM de escritura para que Olly pare en el momento de escribir en ella, de esta manera parará justo antes de terminar el trabajo con la tabla:



Ahora sí está todo preparado, pero cuidado que estamos en una excepción parados tenedlo en cuenta, en lugar de pulsar F9 para dejar correr el programa deberemos pulsar Shift+F9! sinó caeremos en un error y no correrá, superamos la excepción y...



Vaya con esto no contábamos! un error ¿? si hemos hecho todo bien.. incluso repetí todo el proceso a ver si es que me había equivocado en algo pero seguía saliendo este mismo error, es aquí cuando me puse a investigar lo que pasaba, pues en ASProtect's más modernos no ocurre esto, no nos queda más remedio que investigar el error...

Encontrando el error...

Es en este punto cuando nos tenemos que meter más en el packer que nunca, debemos entender lo que está pasando para poder evitar el fallo que nos tiene atrapados, una posible solución sería tracear a mano algunas entradas, ir viendo lo que hace hasta llegar al punto donde nos saca el mnsaje, reiniciamos Olly por enésima vez ;-) y llegamos a la primera excepción, tildamos las excepciones para que no pare en ellas y hacemos un "Go" a la dirección del mágico, ponemos un BP sobre él y pasamos la excepción con Shift+F9, caeremos sobre el CALL, desde aquí nos ponemos a tracear mirando como va rellenando la IAT, enseguida llegamos a un loop donde va tratando todas las entradas, lo vemos en la siguiente imagen:

```
Address   Hex dump        Disassembly                      Comment
00DBF407    87FE          XCHG ESI,EDI
00DBF409    AC            LODS BYTE PTR DS:[ESI]           ;inicio del loop..
00DBF40A    3A05 8427DC00 CMP AL,BYTE PTR DS:[DC2784]
00DBF410  ^ 74 DD         JE SHORT 00DBF3EF
00DBF412    4E            DEC ESI
00DBF413    53            PUSH EBX
00DBF414    56            PUSH ESI
00DBF415    53            PUSH EBX
00DBF416    8D5D F4       LEA EBX,DWORD PTR SS:[EBP-C]
00DBF419    53            PUSH EBX
00DBF41A    3A05 8627DC00 CMP AL,BYTE PTR DS:[DC2786]
00DBF420  v 74 07         JE SHORT 00DBF429
00DBF422    0FB64E 01     MOVZX ECX,BYTE PTR DS:[ESI+1]
00DBF426    41            INC ECX
00DBF427  v EB 05         JMP SHORT 00DBF42E
00DBF429    B9 04000000   MOV ECX,4
00DBF42E    41            INC ECX
00DBF42F    01CE          ADD ESI,ECX
00DBF431    E8 DEFBFFFF   CALL 00DBF014
00DBF436    5B            POP EBX
00DBF437  ^ EB D0         JMP SHORT 00DBF409               ;al inicio del loop..
00DBF439    61            POPAD
00DBF43A    E8 51000000   CALL 00DBF490
00DBF43F    68 48F4DB00   PUSH 0DBF448
00DBF444    FF0424        INC DWORD PTR SS:[ESP]
00DBF447    C3            RETN
```

en el CALL de :DBF431 es donde realiza todos los trabajos, para cada entrada de la IAT se cumple un ciclo del bucle, podemos ir viendo lo que hace y localizar el error que nos está tirando el trabajo por los suelos, de momento con el método del traceo, bouuufff encontramos que peta al trabajar sobre la entrada [62F540] que curiosamente pertenece a otra DLL y no a las kernel con las que venía trabajando, mirando más a fondo el problema encontré que el error venía de este CALL:

```
Address   Hex dump        Disassembly                      Comment
00DBF0DF  v E9 F8000000   JMP 00DBF1DC
00DBF0E4    E8 01000000   CALL 00DBF0EA                    ;Aqui peta!
00DBF0E9  - E9 83C404FF   JMP FFE0B571
00DBF0EE    35 1BF1DB00   XOR EAX,0DBF11B
```

En ese CALL al que llegamos disparados desde el JE en :0DBF045 que es la zona que decide a qué parte ir si hacia la zona donde está el mágico, hacia esta o hacia una tercera que comienza en :0DBF186, vemos que el mensaje de error aparece en el CALL de la imagen, si entramos dentro con F7 llegamos a este RETN:

```
Address   Hex dump        Disassembly                      Comment
00DBF533    310424        XOR DWORD PTR SS:[ESP],EAX
00DBF536    8B05 1420DC00 MOV EAX,DWORD PTR DS:[DC2014]
00DBF53C    010424        ADD DWORD PTR SS:[ESP],EAX
00DBF53F    C3            RETN
00DBF540    C3            RETN
00DBF541    8D40 00       LEA EAX,DWORD PTR DS:[EAX]
```

Sabemos donde va a saltar, mirando en el stack (en la pila) tenemos almacenada en la posición más baja (donde apunta el registro ESP) la dirección de retorno, veámosla:

```
0012FE00  D9F6EEBA
0012FE04  00DBF11F   <-- Aquí está el error!
0012FE08  00DBF514
0012FE0C  0062F540  cartoon.0062F540
```

Vemos que va a retornar a :D9F6EEBA donde no encontrará NADA y mostrará el mensaje, bien pero si miramos una posición más abajo vemos un valor mucho más normalito :0DBF11F donde sí podríamos retornar con tranquilidad, entonces.. ¿nos ha metido un PUSH de más? eso parece sinó no lo comprendo, si aumenamos ESP en 4 unidades y retornamos hacia la dirección citada (:DBF11F) el programa continúa ejecutándose ¡!

Pero esto en realidad no es correcto, pues ocurre que no desencripta bien esa posición de memoria, si miramos por encima del RETN vemos varios XOR que desencriptan precisamente [ESP] por lo que no podemos hacer mucho ante este problema ;-(

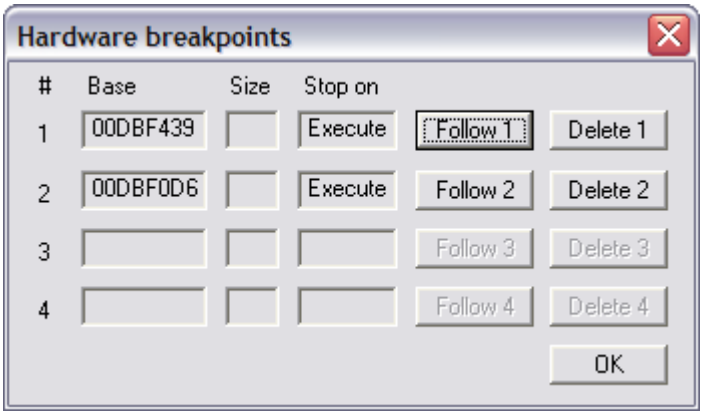
Además me he fijado que aunque no toquemos el mágico con sólo poner un BP en cualquier instrucción no desencripta bien este valor, por esto creo que hace un chequeo CRC de la zona y con el valor obtenido desencripta, esto repito es una suposición personal.

Nos programamos un pequeño script para recuperar parte de la iat a ver qué es concretamente lo que debemos reparar, y nos ayudará a saber concretamente lo que está MAL, podmose usar este mismo:

```
-----------// inicio_script
inicio:
eob vamos                ; si hay una interrupción/excepción salta a "vamos"
run                      ; dejamos correr el programa

vamos:                   ; inicio etiqueta "vamos"
cmp eip, 0DBF0D6         ; estamos en el mágico?
jne no_es                ; si no es así vamos a "no_es"
mov eip, 0DBF0DB         ; si estamos, entonces modificamos EIP (nopeo call simulado)
jmp inicio               ; y seguimos procesando entradas

no_es:                   ; inicio etiqueta "no_es"
cmp eip, 0DBF439         ; estamos en el POPAD?
je termina               ; si, pues hemos terminado ;-)
jmp inicio               ; no, entonces continúa procesando entradas

termina:                 ; etiqueta "termina"
ret                      ; final de script
-----------// final_script
```

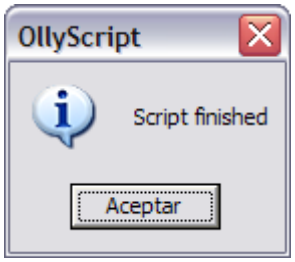
bien simple, lo que hace únicamente es "nopear" el mágico de una manera muy sutil tal y como le hacíamos al Armadillo, de manera que no detectará el cambio y el supuesto CRC queda anulado y hacemos la misma función que nopeando pero sin cambiar ningún byte, primero debemos escorger el punto gatillo desde el cual el scrpt será lanzado, esto es fácil, lo haremos desde el mismo mágico, el segundo HE (el de :0DBF439) lo utilizamos como freno, si miramos el código nos daremos cuenta de que por aquí sólo pasaremos cuando haya terminado el trabajo de la IAT, y ese es buen momento para engancharlo y llegar al OEP con todo semi-resuelto, metemos estas líneas en un archivo de texto llmado "script.txt" reiniciamos Olly, estando en el EP quitamos la tildes de las casillas del cuadro de excepciones (queremos que Olly pare) damos RUN y efectivamente para en la primera de ellas, colocamos nuevamente las tildes en las excepciones, pulsamos Ctr+G y le damos la dirección del mágico :00DBF0D6, le ponemos un BP y superamos la excepción con Shift+F9, caeremos en el mágico, quitamos el BP y le colocamos un HE al igual que en el POPAD que hay en :00DBF439, hecho esto podemos ejecutar el script desde "Plugins->OllyScript->Run script..."

Los HE que debemos tener puestos para que funcione son estos dos:



Si no están ambos puestos el script no funcionará tenedlo en cuenta! es debido a que algunas etiquetas tomarán el control sólo en el caso de que Olly pare en un BreakPoint (del tipo que sea: BP, HE, etc..) y la dirección sea la que nosotros hayamos escogido, si no tenemos ningún BP/HE.. en la dirección elegida nunca parará allí y por

consiguiente nunca se ejecutará el código que contienen la etiqueta, ok? cuando termine su trabajo mostrará este mensajito...



Buena señal, el script se ha ejecutado sin problemas, si nos fijamos estamos precisamente en el POPAD donde queríamos estar (nuestro freno):

```
Address   Hex dump     Disassembly                    Comment
00DBF42F    01CE       ADD ESI,ECX
00DBF431    E8 DEFBFFFF CALL 00DBF014
00DBF436    5B         POP EBX
00DBF437  ^ EB D0      JMP SHORT 00DBF409
00DBF439    61         POPAD          <---- Estamos aquí...
00DBF43A    E8 51000000 CALL 00DBF490
00DBF43F    68 48F4DB00 PUSH 0DBF448
00DBF444    FF0424     INC DWORD PTR SS:[ESP]
00DBF447    C3         RETN
```

Allí quedamos parados en :0DBF439 pero falta algo: llegar al OEP, los pasos son los siguientes: activamos las excepciones (quitando las tildes...) pulsamos F9 para que corra el programa, irá parando en las excepciones que se van sucediendo, llegamos a la última, vamos a la ventanita de memoria Alt+M y colocamos un BPM en la sección de código (la siguiente al header) superamos la excepción Shift+F9 y estamos en el OEP ;-)

```
Address   Hex dump     Disassembly                    Comment
00401000  EB 10        JMP SHORT cartoon.00401012
00401002  66:623A      BOUND DI,DWORD PTR DS:[EDX]
00401005  43           INC EBX
00401006  2B2B         SUB EBP,DWORD PTR DS:[EBX]
00401008  48           DEC EAX
00401009  4F           DEC EDI
0040100A  4F           DEC EDI
0040100B  4B           DEC EBX
0040100C  90           NOP
0040100D  E9 68665D00  JMP 009D767A
00401012  A1 5B665D00  MOV EAX,DWORD PTR DS:[5D665B]
00401017  C1E0 02      SHL EAX,2
0040101A  A3 5F665D00  MOV DWORD PTR DS:[5D665F],EAX
0040101F  52           PUSH EDX
00401020  6A 00        PUSH 0
00401022  E8 FF381D00  CALL cartoon.005D4926          JMP to kernel32.GetModuleHandleA
00401027  8BD0         MOV EDX,EAX
00401029  E8 DA731A00  CALL cartoon.005A8408
0040102E  5A           POP EDX
0040102F  E8 38731A00  CALL cartoon.005A836C
00401034  E8 0F741A00  CALL cartoon.005A8448          Importaciones reparadas..
00401039  6A 00        PUSH 0
0040103B  E8 78101B00  CALL cartoon.005B20B8
00401040  59           POP ECX
00401041  68 04665D00  PUSH cartoon.005D6604
00401046  6A 00        PUSH 0
00401048  E8 D9381D00  CALL cartoon.005D4926          JMP to kernel32.GetModuleHandleA
0040104D  A3 63665D00  MOV DWORD PTR DS:[5D6663],EAX
```

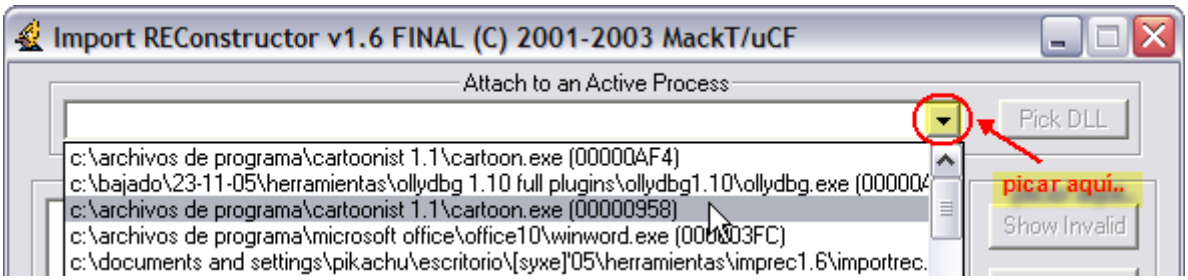
Vemos que la mayoría de importaciones quedaron reparadas gracias al script, esos dos CALL antes no apuntaban a la función GetModuleHandle porque estaba redirigida, si nos situamos sobre alguno de ellos y pulsamos Enter caeremos en la tabla de saltos:

```
Address   Hex dump     Disassembly                    Comment
005D4926  FF25 ECF36200 JMP DWORD PTR DS:[62F3EC]     kernel32.GetModuleHandleA
005D492C  FF25 F0F36200 JMP DWORD PTR DS:[62F3F0]     kernel32.GetOEMCP
005D4932  FF25 F4F36200 JMP DWORD PTR DS:[62F3F4]     kernel32.GetPrivateProfileStringA
005D4938  FF25 F8F36200 JMP DWORD PTR DS:[62F3F8]     <----
005D493E  FF25 FCF36200 JMP DWORD PTR DS:[62F3FC]     kernel32.GetProcessHeap
005D4944  FF25 00F46200 JMP DWORD PTR DS:[62F400]     kernel32.GetProfileStringA
005D494A  FF25 04F46200 JMP DWORD PTR DS:[62F404]     kernel32.GetStartupInfoA
005D4950  FF25 08F46200 JMP DWORD PTR DS:[62F408]     kernel32.GetStdHandle
005D4956  FF25 0CF46200 JMP DWORD PTR DS:[62F40C]     kernel32.GetStringTypeA
005D495C  FF25 10F46200 JMP DWORD PTR DS:[62F410]     kernel32.GetStringTypeW
005D4962  FF25 14F46200 JMP DWORD PTR DS:[62F414]     kernel32.GetSystemDefaultLangID
005D4968  FF25 18F46200 JMP DWORD PTR DS:[62F418]     kernel32.GetSystemInfo
005D496E  FF25 1CF46200 JMP DWORD PTR DS:[62F41C]     kernel32.GetSystemTime
005D4974  FF25 20F46200 JMP DWORD PTR DS:[62F420]     kernel32.GetThreadLocale
005D497A  FF25 24F46200 JMP DWORD PTR DS:[62F424]     kernel32.GetTickCount
005D4980  FF25 28F46200 JMP DWORD PTR DS:[62F428]     kernel32.GetTimeZoneInformation
```

Tiene un aspecto bastante mejorado en comparación con la última vez que la vimos donde no aparecía el nombre de ninguna api al lado, pero ahora los tenemos casi todos, muy buena señal, aunque aún sigue habiendo punteros inválidos como el señalado en la imagen en [62F3F8] que todavía no apunta a ninguna api, esto por su puesto lo debemos reparar para que corra bien el dumpeado sinó no tendrá lógica lo que estamos haciendo, hagamos un breve balance de la situación..

Midiendo Fuerzas...

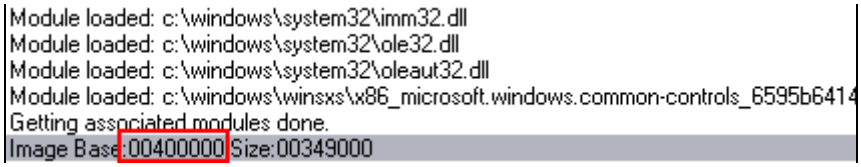
Lo siguiente que haremos será valorar como quedó la tabla de importaciones tras nuestra incursión, y qué mejor manera de hacerlo que con la ayuda del Import Reconstructor 1.6 una genial herramienta de MackT/uCF que nos permite crear una IT en el proceso dumpeado, a partir de esta se creará después la IAT en tiempo de ejecución, pues bueno lo que debemos hacer es primero de todo attachear el programa orignial de la memoria:



Una vez seleccionado de la lista quedará automáticamente attacheado, le damos entonces los valores correctos que son estos:

OEP: aquí ponemos el offset del OEP
RVA: Aquí ponemos el offset del comienzo de la IAT
Size: Ponemos el largo (longitud) de la IAT

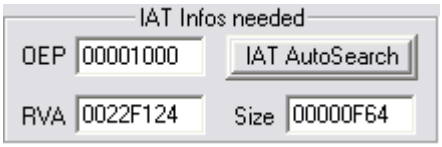
Conociendo estos datos es fácil obtener su offset, sólonos hace falta saber el valor del ImageBase que nos lo da el propio ImpRec:



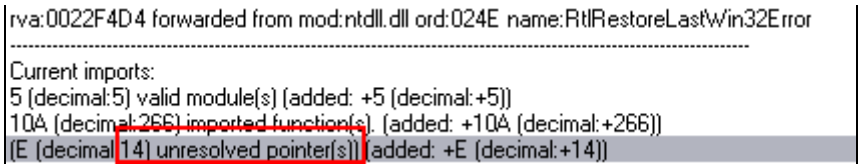
Vemos que es 400000 como en la mayoría de ejecutables, ahora sí que podemos obtener los datos necesarios, partimos de estos:

```
OEP: 401000 -> RVA=401000-400000=1000
RVA (inicio_iat): 62F124 -> RVA=62F124-400000=22F124
Size (tamaño_iat): F64
```

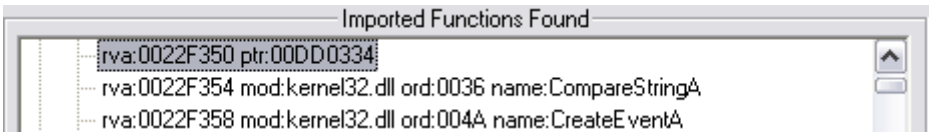
Vamos al ImpRec y rellenamos las casillas con estos valores:



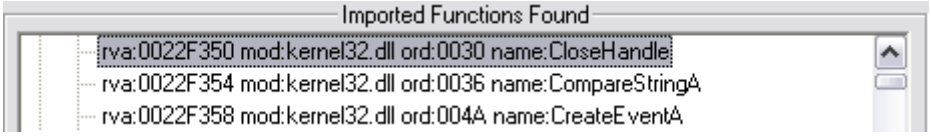
Pulsamos "Get Imports" y nos dirá cuantas entradas quedaron por resolver:



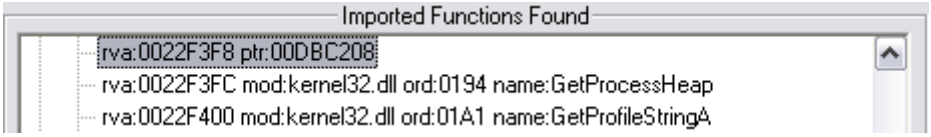
Bueno pues 14h que serían 20d entradas las que tenemos por resolver todavía, y esto gracias al script que si no serían seguro más de 300, jeje.. el siguiente punto pasa por ver si hay alguna que podamos resolver con el plugin del Import, esto ocurre con la entrada que vemos a continuación:



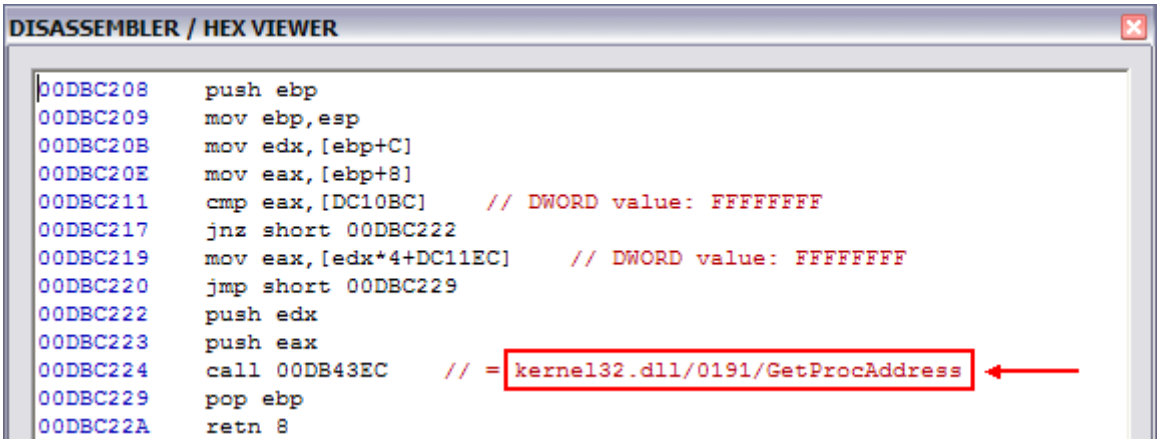
Picamos con el botón derecho sobre ella y elegimos "Trace Level1 (Disasm)" y mirad:



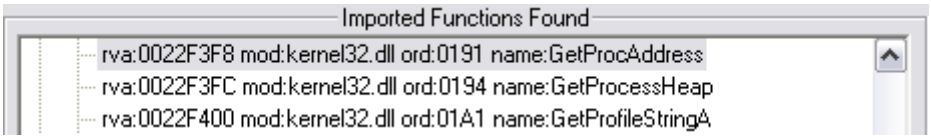
Ya nos queda una menos ;-) en este caso CloseHandle era la api correcta para esta entrada, podríamos hacer esto con las 19 restantes pero desgrasciadamente no funciona, hay algunas que no se reparan tan fácil, otra que sí podemos reparar es ésta:



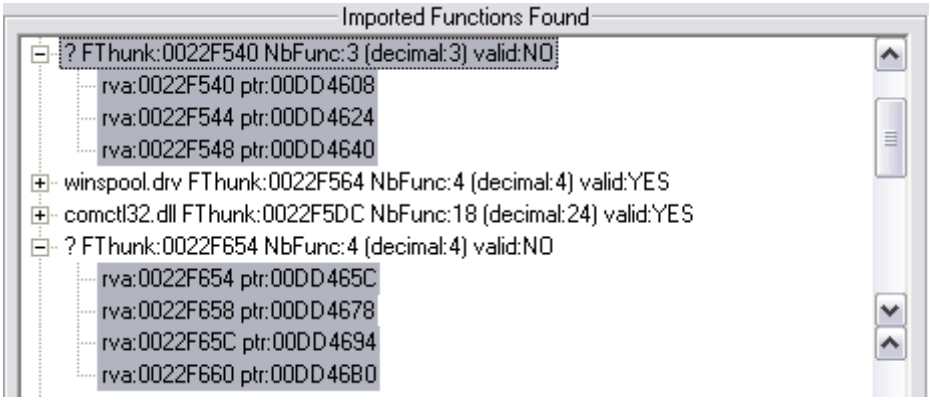
Esta vez la reparamos con el plugin y no con el Tracer, para ver de qué api se trata picamos sobre ella y elegimos "Disassemble/HexView" y nos muestra esto:



Allí vemos que la api correcta es GetProcAddress, cerramos este cuadro y hacemos doble clic sobre la entrada, en el nuevo cuadro que se desplega elegimos esa API y queda resuelta, jeje.. otra menos, también la podríamos reparar con el plugin para el ASProtect 1.22 que también funciona, igual da, lo importante es que nos quede esto:



Sólo nos quedan 18 ;-) pero el resto son todos thunks enteros, es decir apis de la misma librería y no hay modo alguno de repararlas desde el Import, veamos cuales son:



Estos son dos de ellos el tercero es el siguiente:

```
Imported Functions Found
? FThunk:0022F998 NbFunc:5 (decimal:5) valid:NO
    rva:0022F998 ptr:00DD46CC
    rva:0022F99C ptr:00DD46E8
    rva:0022F9A0 ptr:00DD4704
    rva:0022F9A4 ptr:00DD4720
    rva:0022F9A8 ptr:00DD473C
user32.dll FThunk:0022FCB4 NbFunc:C0 (decimal:192) valid:YES
```

Y con este queda toda la iat revisada, al final nos queamos con estas entradas por resolver, y como digo el ImpRec es incapaz de ayudarnos más, el resto es tarea nuestra, aquí cuando viene el estudio de ASProtect, he de decir que le di muchas vueltas a este tema buscando una posible solución, lo primero que hice fue tracear línea por línea al packer para lograr comprender su funcionamiento, y poder desmantelarlo, esto fue lo que fui viendo:

Nada más ejecutar el programa llego hasta el mágico (ya sabemos cómo) una vez allí comienzo a tracear con F7, sí con F7 habéis oído bien (sin prisas, jeje..) y examinar en detalle lo que hace, así fueron muchas las cosas que vi, algunas de ellas (las más relevantes) son éstas:

pero antes revisemos el loop donde se encarga de rellenar la IAT:

```
Address   Hex dump     Disassembly                    Comment
00DBF407    87FE       XCHG ESI,EDI
00DBF409    AC         LODS BYTE PTR DS:[ESI]         ;obtiene el identificador..
00DBF40A    3A05 8427DC00  CMP AL,BYTE PTR DS:[DC2784] mira si es "5A"..
00DBF410  ^ 74 DD      JE SHORT 00DBF3EF              ;si salta cambia de DLL..
00DBF412    4E         DEC ESI                        ;ESI apunta al identificador de api..
00DBF413    53         PUSH EBX                       ;puntero a DLL tratada + id de DLL..
00DBF414    56         PUSH ESI                       ;puntero a Id de api..
00DBF415    53         PUSH EBX                       ;puntero a DLL tratada + id de DLL..
00DBF416    8D5D F4    LEA EBX,DWORD PTR SS:[EBP-C]   ;toma un puntero a puntero de tabla..
00DBF419    53         PUSH EBX                       ;lo salva..
00DBF41A    3A05 8627DC00  CMP AL,BYTE PTR DS:[DC2786] ;CMP Id de api con "6F"..
00DBF420  v 74 07      JE SHORT 00DBF429
00DBF422    0FB64E 01  MOVZX ECX,BYTE PTR DS:[ESI+1]  ;toma longitud de nombre de la api..
00DBF426    41         INC ECX                        ;le suma 1 unidad..
00DBF427  v EB 05      JMP SHORT 00DBF42E
00DBF429    B9 04000000  MOV ECX,4
00DBF42E    41         INC ECX                        ;le suma otra ud..
00DBF42F    01CE       ADD ESI,ECX                    ;ESI apuntara al final del nombre api..
00DBF431    E8 DEFBFFFF  CALL 00DBF014                ;realiza todo el trabajo..
00DBF436    5B         POP EBX                        ;puntero a la DLL tratada..
00DBF437  ^ EB D0      JMP SHORT 00DBF409             ;y salta al inicio del loop..
```

Al lado añado algunos comentarios de lo que va haciendo, pero antes de continuar debo aclarar algunos conceptos, el valor "Id api" o "Identificador de api" quiere decir lo siguiente: el packer tiene una tabla donde guarda el nombre de la DLL (p.ej. kernel32.dll) y al lado el nombre de las apis que utilizará de esta librería, pero este nombre (el de las apis) está encriptado (no así el de la .DLL que se puede leer bien) ahora, esta tabla tiene un formato específico para poder leerla, es aquí cuando cobra valor el "Id de api" que se corresponde siempre con el valor "4Ah" Para comprender mejor todo esto nos situamos sobre la línea :0DBF409 donde va a leer de [ESI] allí encontraremos mediante un "Follow in dump->Memory Address" la citada tabla:

```
Address   Hex dump                                          ASCII
00DD25CC  01 68 6F 6F 6B 64 6C 6C 2E 64 6C 6C 00 4A 0E F2  @hookdll.dll.J♫≡
00DD25DC  6B 4A D8 D9 70 82 0B 9D F9 47 88 4D 05 4A 10 EE  kJÏ┘pé∂Ø¨GêM♣J►∞
00DD25EC  6B 70 C2 CB 68 8F 2C B3 FA 6A AF 49 6A 85 E1 5A  kp┬╦hÅ,│·j»IjàßZ
00DD25FC  50 F1 22 00 01 61 64 76 61 70 69 33 32 2E 64 6C  P±".@advapi32.dl
00DD260C  6C 00 4A 0C E9 60 5E EF D4 73 9D 25 94 D4 51 E7  l.J.ú'^'ésØ%öëQ╠
00DD261C  4A 10 E9 60 5E EF CA 79 8F 34 BA FA 4D 9E 63 7D  J►ú'^'╧yÅ4│·M×c}
00DD262C  AF E1 4A 0E E9 60 5E E8 DD 70 8B 34 BA FA 4D 9E  »ßJ♫ú'^è¦pï4│·M×
00DD263C  67 05 4A 0C E9 60 5E EA D4 69 9D 28 94 D4 51 E7  g♣J.ú'^ûêiØ(öëQ╠
00DD264C  4A 0E E9 60 5E E3 C8 79 80 0B BA C8 6D 9F 67 05  J♫ú'^ⁿ╚yÇ∂│ºmƒg♣
00DD265C  4A 11 E9 60 5E FD CD 79 9C 39 89 D0 44 92 43 40  J◄ú'^²═y£9ë$DÆÉC@
00DD266C  96 A0 1E 4A 0F E9 60 5E FF DD 68 B8 21 B3 C4 4D  ûá▲J☼ú'^ ¡h@!│─M
00DD267C  A2 5E 44 EE 5A 50 F3 22 00 02 6B 65 72 6E 65 6C  ó^D¯ZP%".☻kernel
00DD268C  33 32 2E 64 6C 6C 00 4A 0C CE 55 E6 2A BE 09 0C  32.dll.J.╬Uµ*¥..
00DD269C  BA 29 DF 48 88 4A 0F CE 56 E4 29 BA 33 08 87 39  │)▀HêJ☼╬V∞)│3◘ç9
00DD26AC  C1 44 E6 9E 4E 28 4A 0D CE 4B EC 38 AF 24 28 A2  ┴DµxN(J.╬Kÿ8»$(ó
00DD26BC  28 DD 59 C9 F9 4A 0C CE 4B EC 38 AF 24 2B BD 21  (¦Y╔·J.╬Kÿ8»$+¢!
00DD26CC  D6 6C 88 4A 13 CE 4B EC 38 AF 24 2B BD 21 D6 60  ïlêJ‼╬Kÿ8»$+¢!ï'
00DD26DC  E9 89 7F 41 10 88 C2 6A 4A 0C CE 4B EC 38 AF 24  ûëΩA►ê┬jJ.╬Kÿ8»$
00DD26EC  2B BD 21 D6 7A 88 4A 0D CE 4B EC 38 AF 24 20 A1  +¢!ïzêJ.╬Kÿ8»$ ¡
00DD26FC  39 D6 55 C9 F9 4A 0D CE 4B EC 38 AF 24 39 BC 3F  9ïU╔·J.╬Kÿ8»$9¼?
```

Los valores resaltados son los importantes de ésta (Identificadores, etc.,) siendo:

nombre de la DLL.

identificador de DLL.

identificador de api.

nombre de la api.

separación entre DLL's.

Bueno pues estos son los colores que he utilizado para diferenciar las partes de la tabla, que como vemos está bien estructurada, lo tiene todo bajo control, jeje.. bueno casi todo pero con los CracksLatinoS no contaba ;-)))

Ahora pasaré a detallar qué son cada uno de estos identificadores:

1.- Nombre de la DLL: No hay problema, se entiende bien.
2.- Identificador de DLL: Este valor acompaña al nombre de la misma, suele ser un 01, 02, 04, etc., he estudiado bien la protección y llegué a la conclusión de que este valor se utiliza en este CALL:

```
00DBF038    E8 2B37FFFF     CALL 00DB2768            ; se utiliza Aquí!
00DBF03D    80EB 02         SUB BL,2
00DBF040    74 0E           JE SHORT 00DBF050
00DBF042    80EB 02         SUB BL,2
00DBF045    0F84 99000000   JE 00DBF0E4
00DBF04B    E9 36010000     JMP 00DBF186
```

Al final en función del identificador de DLL que retorna en BL saltará o no en :0DBF040, :0DBF045 ó :0DBF04B, intuyo que este valor le indica al packer "la manera" de tratar a dicha librería, por ejemplo las del kernel las trata distinto que las de la version, etc.,
3.- Identificador de api: Se utiliza exclusivamente para delimitar la tabla, cuando el packer encuentra el valor "4A" sabe que apartir de allí comienza una api.
4.- Nombre de la api: Este tampoco tiene ningún misterio, debemos saber que el nobre de la misma va encriptado sino de qué valdría redirigir las apis ¿? jeje.. está claro verdad? después veremos cómo y donde desencripta este nombre.
5.- Separación entre DLL's: este valor "5A" le indica al packer que la DLL terminó y apartir de este momento se empezarán atratar apis de otra librería.

Y hasta aquí el detalle de las partes más importantes, ahora vamos a lo práctico, tracearemos el loop en el que ASProtect rellena y redirige la iat en busca de datos que nos ayuden a solucionar el tema, llegamos al call semi-mágico y desde ahí comenzamos, al poco llegaremos al citado bucle donde empezaremos a estudiarlo en detalle, entramos en el "CALL 0DBF014" y vemos como va funcionando, llega a la zona que vimos justo antes donde se mete en el "CALL 00DB2768" y toma el Identificador de DLL, en función del valor hará un salto ,hacia 0DBF050, 0DBF0E4 ó 0DBF186, la primera de ellas (0DBF050) nos lleva a la zona del call mágico, esto lo hace para algunas librerías como el kernel, user, etc., si estudiamos esta zona vemos esto:

```
Address   Hex dump      Disassembly                 Comment
00DBF04B  E9 36010000  JMP 00DBF186
00DBF050  8A07         MOV AL,BYTE PTR DS:[EDI]    ;pone el Identificador de api en AL..
00DBF052  3A05 8627DC00 CMP AL,BYTE PTR DS:[DC2786] ;mira si es "6F"..
00DBF058  74 3C        JE SHORT 00DBF096
00DBF05A  8D47 01      LEA EAX,DWORD PTR DS:[EDI+1] ;pone en EAX la direccion del Id. api+1..
00DBF05D  8A00         MOV AL,BYTE PTR DS:[EAX]    ;pone longitud del nombre api en AL..
00DBF05F  8845 E3      MOV BYTE PTR SS:[EBP-1D],AL ;y salva este valor..
00DBF062  8D5F 02      LEA EBX,DWORD PTR DS:[EDI+2] ;EBX apunta al nombre de api encriptado..
00DBF065  33C9         XOR ECX,ECX                 ;pone ECX a cero..
00DBF067  8A4D E3      MOV CL,BYTE PTR SS:[EBP-1D] ;pone en CL longitud nombre api..
00DBF06A  8D85 E3FEFFFF LEA EAX,DWORD PTR SS:[EBP-11D] ;EAX apunta donde escribira nombre api..
00DBF070  8BD3         MOV EDX,EBX
00DBF072  E8 D553FFFF  CALL 00DB444C               ;escribe el nombre de la api encriptado..
00DBF077  6A 0A        PUSH 0A
00DBF079  B9 6E27DC00  MOV ECX,0DC276E
00DBF07E  33D2         XOR EDX,EDX
00DBF080  8A55 E3      MOV DL,BYTE PTR SS:[EBP-1D] ;pone en DL longitud nombre api..
00DBF083  8D85 E3FEFFFF LEA EAX,DWORD PTR SS:[EBP-11D] ;EAX apunta donde escribira nombre api..
00DBF089  E8 66D0FFFF  CALL 00DBC0F4               ;desencripta nombre de la api..
00DBF08E  8D9D E3FEFFFF LEA EBX,DWORD PTR SS:[EBP-11D] ;EBX apuntara nombre api desencriptado..
00DBF094  EB 05        JMP SHORT 00DBF09B          ;y sigue..
00DBF096  8D47 01      LEA EAX,DWORD PTR DS:[EDI+1]
```

Lo que hace es bien sencillo, primero prepara los registros para operar y reserva una zona de memoria donde quedará desencriptado el nombre de la api que corresponda, podemos verlo un poco más detallado:

```
Address   Hex dump        Disassembly                  Comment
00DBF04B  E9 36010000     JMP 00DBF186
00DBF050  8A07            MOV AL,BYTE PTR DS:[EDI]     ;pone el Identificador de api en AL..
00DBF052  3A05 8627DC00   CMP AL,BYTE PTR DS:[DC2786]  ;mira si es "6F"..
00DBF058  74 3C           JE SHORT 00DBF096
```

Aquí lo que hace es leer de la famosa tabla el valor correspondiente a [EDI] y mira si es igual a 6Fh,si es este momento hacemos un "Follow in dump->Memory Address" veremos justamente de donde está leyendo:

```
Address   Hex dump                                         ASCII
00DD26B2  4A 0D CE 4B EC 38 AF 24 28 A2 28 DD 59 C9 F9 4A  J♪╬Kм8»$(ó(¦Y╔¨J
00DD26C2  0C CE 4B EC 38 AF 24 2B BD 21 D6 6C 88 4A 13 CE  ♀╬Kм8»$+¢!╓lêJ‼╬
00DD26D2  4B EC 38 AF 24 2B BD 21 D6 60 E9 89 7F 41 10 88  Kм8»$+¢!╓`ΘëΔ►êê
00DD26E2  C2 6A 4A 0C CE 4B EC 38 AF 24 2B BD 21 D6 7A 88  ┬jJ♀╬Kм8»$+¢!╓zê
```

lee justamente el "4A" que es el Id. api como le pusimos antes, justo después se ve el tamaño de la misma "0D" y luego el nombre encriptado, si resulta que coinciden ambos valores comparados saltará hacia :0DBF096 evitando la desencriptación de la api, hasta ahora no he descubierto qué significa este 6Fh pero tampoco importa demasiado por lo que veo pues casi nunca salta (por no decir nunca).

Lo siguiente que hace es preparar los registros y la zona de memoria donde escribirá el nombre de la api (encriptado de momento):

```
00DBF05A  8D47 01         LEA EAX,DWORD PTR DS:[EDI+1]   ;pone en EAX la direccion del Id. api+1..
00DBF05D  8A00            MOV AL,BYTE PTR DS:[EAX]       ;pone longitud del nombre api en AL..
00DBF05F  8845 E3         MOV BYTE PTR SS:[EBP-1D],AL    ;y salva este valor..
00DBF062  8D5F 02         LEA EBX,DWORD PTR DS:[EDI+2]   ;EBX apunta al nombre de api encriptado..
00DBF065  33C9            XOR ECX,ECX                    ;pone ECX a cero..
00DBF067  8A4D E3         MOV CL,BYTE PTR SS:[EBP-1D]    ;pone en CL longitud nombre api..
00DBF06A  8D85 E3FEFFFF   LEA EAX,DWORD PTR SS:[EBP-11D] ;EAX apunta donde escribira nombre api..
```

Si miramos los registros veremos los valores correspondientes a cada entrada, esto lo evitamos de momento sino el tute se hará eterno ;-) y por fin escribe la api:

```
00DBF070  8BD3            MOV EDX,EBX
00DBF072  E8 D553FFFF     CALL 00DB444C                  ;escribe el nombre de la api encriptado..
```

Si miramos en la ventana dump allí veremos la api todavía encriptada:

```
Address   Hex dump                                         ASCII
0012FE1B  CE 4B EC 38 AF 24 28 A2 28 DD 59 C9 F9 00 00 00  ╬Kм8»$(ó(¦Y╔¨...
0012FE2B  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0012FE3B  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0012FE4B  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Allí la vemos, pero todavía falta desencriptarla claro, esto lo hace justo después:

```
00DBF077  6A 0A           PUSH 0A
00DBF079  B9 6E27DC00     MOV ECX,0DC276E
00DBF07E  33D2            XOR EDX,EDX
00DBF080  8A55 E3         MOV DL,BYTE PTR SS:[EBP-1D]    ;pone en DL longitud nombre api..
00DBF083  8D85 E3FEFFFF   LEA EAX,DWORD PTR SS:[EBP-11D] ;EAX apunta donde escribira nombre api..
00DBF089  E8 66D0FFFF     CALL 00DBC0F4                  ;desencripta nombre de la api..
00DBF08E  8D9D E3FEFFFF   LEA EBX,DWORD PTR SS:[EBP-11D] ;EBX apuntara nombre api desencriptado..
00DBF094  EB 05           JMP SHORT 00DBF09B             ;y sigue..
00DBF096  8D47 01         LEA EAX,DWORD PTR DS:[EDI+1]
```

como antes prepara los registros para que contengan los valores necesarios, después pone en EBX un puntero a la api encriptada y se mete en el CALL donde por fin desencripta, por curiosidad decir que desencripta en este loop:

```
Address   Hex dump        Disassembly                  Comment
00DB8BBB  D1E9            SHR ECX,1
00DB8BBD  49              DEC ECX                      ;decrementa el contador..
00DB8BBE  7C 0B           JL SHORT 00DB8BCB            ;salta si hemos terminado..
00DB8BC0  8B048E          MOV EAX,DWORD PTR DS:[ESI+ECX*4]  ;lee de la memoria..
00DB8BC3  33048A          XOR EAX,DWORD PTR DS:[EDX+ECX*4]  ;desencripta el valor leÿdo..
00DB8BC6  89048F          MOV DWORD PTR DS:[EDI+ECX*4],EAX  ;y lo almacena en memoria..
00DB8BC9  EB F2           JMP SHORT 00DB8BBD           ;vuelta al inicio..
00DB8BCB  5F              POP EDI
```

En este bucle desencripta la api completamente, podemos verlo claro, nos situamos en la línea :0DB8BC6 donde se dispone a mover el valor de EAX a la memoria y hacemos un "Follow in dump->Memory Address" y veremos los bytes a modificar lógicamente todavía encriptados:

```
Address   Hex dump                                         ASCII
0012FE23  28 DD 59 C9 00 00 00 00 00 00 00 00 00 00 00 00  (¦Y╔............
0012FE33  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0012FE43  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Estos son los bytes finales de la api, si superamos la instrucción con F8 y miramos:

```
Address   Hex dump                                          ASCII
0012FE13  00 7C FF 12 00 00 00 5A CE 4B EC 38 AF 24 28 A2   .| ‡...Z╬Kì8»$(ó
0012FE23  65 6E 74 41 00 00 00 00 00 00 00 00 00 00 00 00   entA............
0012FE33  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
0012FE43  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
```

vemos que desencriptó bien los cuatro últimos valores "entA", jeje.. si seguimos con el procedimiento al final lo veremos completo:

```
Address   Hex dump                                          ASCII
0012FE1B  43 72 65 61 74 65 45 76 65 6E 74 41 00 00 00 00   CreateEventA....
0012FE2B  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
0012FE3B  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
```

Pues esta era CreateEventA, si ahora retornamos de la llamada en la que estamos metidos saldremos en la parte en que estábamos antes y continúa haciendo cosas:

```
Address   Hex dump        Disassembly                  Comment
00DBF07E    33D2          XOR EDX,EDX
00DBF080    8A55 E3       MOV DL,BYTE PTR SS:[EBP-1D]  ;pone en DL longitud nombre api..
00DBF083    8D85 E3FEFFFF LEA EAX,DWORD PTR SS:[EBP-11D] ;EAX apunta donde escribira nombre api..
00DBF089    E8 66D0FFFF   CALL 00DBC0F4                ;desencripta nombre de la api..
00DBF08E    8D9D E3FEFFFF LEA EBX,DWORD PTR SS:[EBP-11D] ;EBX apuntara nombre api desencriptado..
00DBF094  v EB 05         JMP SHORT 00DBF09B           ;y sigue..
00DBF096    8D47 01       LEA EAX,DWORD PTR DS:[EDI+1]
00DBF099    8B18          MOV EBX,DWORD PTR DS:[EAX]
00DBF09B    8A07          MOV AL,BYTE PTR DS:[EDI]     ;pone el Identificador de api en AL..
00DBF09D    3A05 8727DC00 CMP AL,BYTE PTR DS:[DC2787]  ;mira si es "42"..
00DBF0A3  v 75 07         JNZ SHORT 00DBF0AC
00DBF0A5    B8 08C2DB00   MOV EAX,0DBC208              ;caso GetProcAddress..
00DBF0AA  v EB 2F         JMP SHORT 00DBF0DB           ;salta a guardar directamente..
00DBF0AC    3A05 8827DC00 CMP AL,BYTE PTR DS:[DC2788]  ;mira si es "A2"..
00DBF0B2  v 75 17         JNZ SHORT 00DBF0CB
00DBF0B4    53            PUSH EBX                     ;no llega nunca..
00DBF0B5    8B45 0C       MOV EAX,DWORD PTR SS:[EBP+C]
00DBF0B8    40            INC EAX
00DBF0B9    50            PUSH EAX
00DBF0BA    E8 D1FAFFFF   CALL 00DBEB90
00DBF0BF    A3 6825DC00   MOV DWORD PTR DS:[DC2568],EAX
00DBF0C4    B8 30C2DB00   MOV EAX,0DBC230
00DBF0C9  v EB 10         JMP SHORT 00DBF0DB
00DBF0CB    53            PUSH EBX                     ;salva el nombre de la api..
00DBF0CC    8B45 0C       MOV EAX,DWORD PTR SS:[EBP+C] ;toma el Id de DLL..
00DBF0CF    40            INC EAX                      ;incrementa EAX en 1..
00DBF0D0    50            PUSH EAX                     ;salva el nombre de la DLL..
00DBF0D1    E8 BAFAFFFF   CALL 00DBEB90                ;pone la api en EAX..
00DBF0D6    E8 79FEFFFF   CALL 00DBEF54                ;nos chafa el valor de EAX ;-(
00DBF0DB    8B16          MOV EDX,DWORD PTR DS:[ESI]   ;en EDX la posicion de la IAT..
00DBF0DD    8902          MOV DWORD PTR DS:[EDX],EAX   ;y guarda el valor de EAX..
00DBF0DF  v E9 F8000000   JMP 00DBF1DC
```

retornamos a :0DBF08E donde pone en EBX la dirección de la api, después encontramos un JMP que nos pone directamente en :0DBF09B, donde pone en AL el Id. de api que luego compara con "42" y "A2" respectivamente, si es el 42 no salta y pone 0DBC208 en EAX que resulta ser el valor de GetProcAddress, esto es una redirección especial de esta api, podríamos modificar el script para evitar que lo haga pero no importa ya que desde el ImpRec es fácil de reparar. Cuando terminan las comprobaciones llegamos al fin a la zona del mágico que ya antes hemos explicado.

Ya hemos traceado la zona cuando guarda una api buena y cuando guarda una mala, podemos evitar la redirección pero el mayor problema está a la hora de desencriptar la api, pues aunque cambiemos saltos y demás no llegamos a obligarlo a desencriptarla bien, haremos la prueba para ver que esto es cierto: Vamos a tracear una api de las que tiene por redirigir, no sabemos cual es porque el nombre está encriptado, el gran problema además es que no tiene una llave de desencriptación sinó que va desencriptando cada api con un valor distinto, y este valor tampoco se almacena en la misma posición de memoria, cosa que complica muchísimo el trabajo de desencriptar apis que no conocemos. De momento ponemos un BP en este CALL:

```
00DBF038    E8 2B37FFFF     CALL 00DB2768            ;ponemos el BP aquí!
00DBF03D    80EB 02         SUB BL,2
00DBF040    74 0E           JE SHORT 00DBF050
00DBF042    80EB 02         SUB BL,2
00DBF045    0F84 99000000   JE 00DBF0E4
00DBF04B    E9 36010000     JMP 00DBF186
```

A la salida del CALL tenemos BL==04 que es el Identifiador de DLL, si miramos la tabla que el packer tiene descubrimos que se trata de version.dll cuyo identificador es efectivamente 04h, entonces el primer JE no se ejecuta pero sí el segundo que nos encontramos todavía una parte distinta de la anterior:

Address	Hex dump	Disassembly	Comment
00DBF0E4	E8 01000000	CALL 00DBF0EA	;zona especial..
00DBF0E9	- E9 83C404FF	JMP FFE0B571	
00DBF0EE	35 1BF1DB00	XOR EAX,0DBF11B	
00DBF0F3	E8 01000000	CALL 00DBF0F9	

Aquí se llega desde el segundo JE del código anterior, esta es la zona que se encarga de tratar las DLL's cuyo identificador es 04h, entramos en el CALL y nos encontramos una grata sorpresa:

Address	Hex dump	Disassembly	Comment
00DBF174	B9 16000000	MOV ECX,16	
00DBF179	E8 DA34FFFF	CALL 00DB2658	
00DBF17E	8BC3	MOV EAX,EBX	
00DBF180	8B16	MOV EDX,DWORD PTR DS:[ESI]	;en EDX la posicion IAT..
00DBF182	8902	MOV DWORD PTR DS:[EDX],EAX	;rellena la iat..
00DBF184	EB 56	JMP SHORT 00DBF1DC	;y seguimos..
00DBF186	8A07	MOV AL,BYTE PTR DS:[EDI]	

jeje.. otra zona donde rellena la IAT, o sea que para estas funciones tiene una zona especial donde además rellena la tabla en otro punto diferente, esto si es lo más, si nos fijamos bien vemos que EAX toma su valor de EBX y que éste lo toma aquí:

```
00DBF130    E8 3334FFFF     CALL 00DB2568
00DBF135    8BD8            MOV EBX,EAX       ; EBX toma su valor aqui...
00DBF137    B8 F2FFFFFF     MOV EAX,-0E
00DBF13C    83E8 E8         SUB EAX,-18
00DBF13F    8945 E4         MOV DWORD PTR SS:[EBP-1C],EAX
00DBF142    A0 A814DC00     MOV AL,BYTE PTR DS:[DC14A8]
```

EBX toma el valor en :0DBF135 y lo hace de EAX suponemos que lo hace del CALL anterior, puse un HE en este CALL ya que los BP los detecta y comencé a tracear y cual fue mi sorpresa una vez más? que logré encontrar otra interesante zona, primero pasé por el CALL que desencripta la api y finalmente llegué aquí:

Address	Hex dump	Disassembly	Comment
00DBF1CA	47	INC EDI	
00DBF1CB	8B1F	MOV EBX,DWORD PTR DS:[EDI]	;pone la api en EBX..
00DBF1CD	53	PUSH EBX	;guarda el nombre de la api..
00DBF1CE	8B45 0C	MOV EAX,DWORD PTR SS:[EBP+C]	;pone el Id de la DLL en EAX..
00DBF1D1	40	INC EAX	;le cuma 1 unidad..
00DBF1D2	50	PUSH EAX	;empuja el nombre de la DLL..
00DBF1D3	E8 B8F9FFFF	CALL 00DBEB90	;llama a GetProcAddress..
00DBF1D8	8B16	MOV EDX,DWORD PTR DS:[ESI]	;en EDX la posicion de la IAT..
00DBF1DA	8902	MOV DWORD PTR DS:[EDX],EAX	GDI32.Arc
00DBF1DC	8306 04	ADD DWORD PTR DS:[ESI],4	;apunta a la siguiente entrada..
00DBF1DF	5F	POP EDI	

Localizamos una tercera zona donde rellena la tabla, en este caso vemos una llamada a GetProcAddress y acto seguido la instrucción donde guarda el valor obtenido en EAX (retorno de GetProcAddress) y por tanto la api correcta, este caso trata la api "Arc" de la Gdi32.dll, a esta parte habíamos llegads a través del JMP que hay en :0DBF04B (después de los dos JE) justo a la salida del CALL que recupera el Id. de DLL en BL.

Nos encontramos con tres zonas donde rellena la IAT, cada una en función del Id. de DLL que se valora en el CALL de :0DBF038 luego hay tres saltos (dos JE y un JMP) que nos envían al lugar correcto en cada caso. Vemos además que es el segundo JE el que nos pone en la peor zona de todas y lo toma para tres librerías concretamente:

1.- version.dll
2.- commdlg32.dll
3.- shell32.dll

Estas tres desde aquí no tienen arreglo ya que no hay ningún mágico que nos pueda ayudar, si tratamos de modificar el JE para llegar a la zona buena nos encontramos con que el nombre de la api no se desencripta correctamente y como no sigue un patrón es improbable que demos con la llave de desencriptación adecuada a cada api, recordemos que cada una se desencripta con una llave distinta. En fin, que este ASProtect es un cabronazo con el tema de las apis.

Llegados a este punto la solución pasa por dejar que rompa la tabla e intentar recuperarla una vez ya estemos en el programa. Una manera sencilla de hacerlo sería poner un BP en cada uno de los "JMP DWORD PTR [xxxxxxxx]" y ver a que api salta para insertarla en la iat, es lo que vamos a hacer ;-)

A por la siguiente parte, bouuuffff...!!!!

Recuperando las importaciones (segunda parte)...

Hemos visto la manera que tiene ASProtect de redirigir las apis en esta versión, y no hemos comentado aún nada del caso GetProcAddress del que vimos que ponía el valor 0DBC208 en EAX y arreando, además tenía una zona especial a la que no llegaba cuyo inicio estaba en :0DBF0B4, esto no quiere decir que en otros programas protegidos con esta versión de ASProtect no la utilice, pero no lo hace en este concreto. La idea es llegar al OEP con la tabla semi reparada, utilizando el script anterior para recuperarla en la medida de lo posible, después nos quedará el trabajo de las tres DLL's comentadas: version, commdlg32 y shell32 que son un caso aparte.

Como ya antes hemos comentado la manera llegar al OEP con la tabla semi-correcta nos centraremos únicamente en el caso especial, vamos a tratar de encontrar las apis correctas de las entradas que faltan. Lo primero que haremos será buscar las referencias a estos saltos, nos dirigimos a las entradas redirigidas:

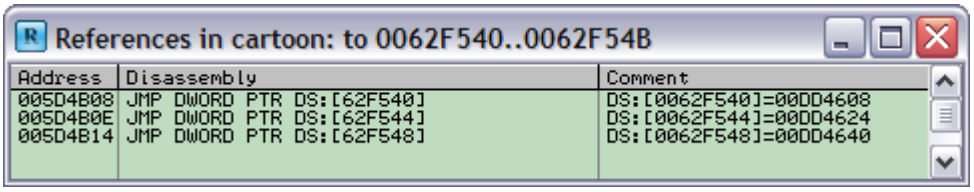
```
Address   Hex dump                                          ASCII
0062F540  08 46 DD 00 24 46 DD 00 40 46 DD 00 00 00 00 00  ◘F¦.$F¦.@F¦.....
0062F550  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0062F560  00 00 00 00 63 16 F8 72 AD 5F F8 72 49 25 F8 72  ....c▬°r↓_°rI%°r
0062F570  69 5E F8 72 00 00 00 00 00 00 00 00 00 00 00 00  i^°r............
0062F580  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0062F590  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0062F5A0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0062F5B0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0062F5C0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0062F5D0  00 00 00 00 00 00 00 00 00 00 00 00 A4 7F 33 77  ............ñ⌂3w
0062F5E0  F7 90 35 77 E3 AD 31 77 3D 51 31 77 38 91 35 77  ≈É5wô↓1w=Q1w8æ5w
0062F5F0  80 91 35 77 5E 91 35 77 9E 91 35 77 B7 4E 31 77  Çæ5w^æ5w×æ5wÀN1w
0062F600  C0 90 35 77 F6 7F 31 77 94 90 35 77 93 5B 31 77  └É5w÷⌂1wöÉ5wô[1w
0062F610  57 A4 31 77 1A 65 33 77 0B 43 33 77 14 93 35 77  Wñ1w→e3w♂C3w¶ô5w
0062F620  25 74 32 77 54 50 31 77 47 90 35 77 A3 93 35 77  %t2wTP1wGÉ5wúô5w
0062F630  59 92 35 77 B1 38 31 77 D4 5B 31 77 00 00 00 00  YÆ5w▒81w╘[1w....
0062F640  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0062F650  00 00 00 00 5C 46 DD 00 78 46 DD 00 94 46 DD 00  ....\F¦.xF¦.öF¦.
0062F660  B0 46 DD 00 00 00 00 00 00 00 00 00 00 00 00 00  ▒F¦.............
0062F670  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Las seleccionadas son las que quedaron por reparar, todavía hay algunas más, recordemos que de las 20 que nos deja hay 2 que se pueden reparar fácil:

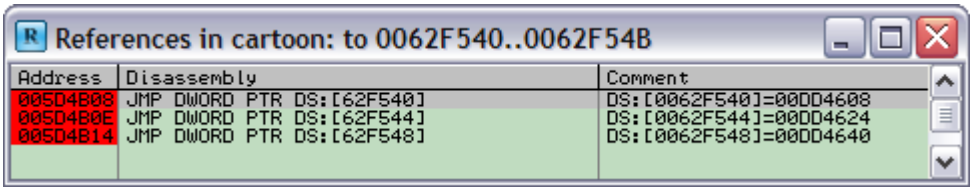
RVA: 22F350 | Apunta a: 0DD0334 | api: CloseHandle |Reparada con: Trace Level1 (Disasm)
RVA: 22F3F8 | Apunta a: 0DBC208 | api: GetProcAddress |Reparada con: plugin ASPr 1.22

Estas dos se recuperan fácilmente, ya antes enseñamos cómo hacerlo más detenidamente, pero aún nos quedan 18 que son las de las tres DLL's antes citadas, vamos a ver como recuperarlas todas ;-)

Lo primero de todo es buscar referencias a las mismas, lo conseguimos seleccionando sus bytes y pulsando Ctrl+R, veremos algo así:



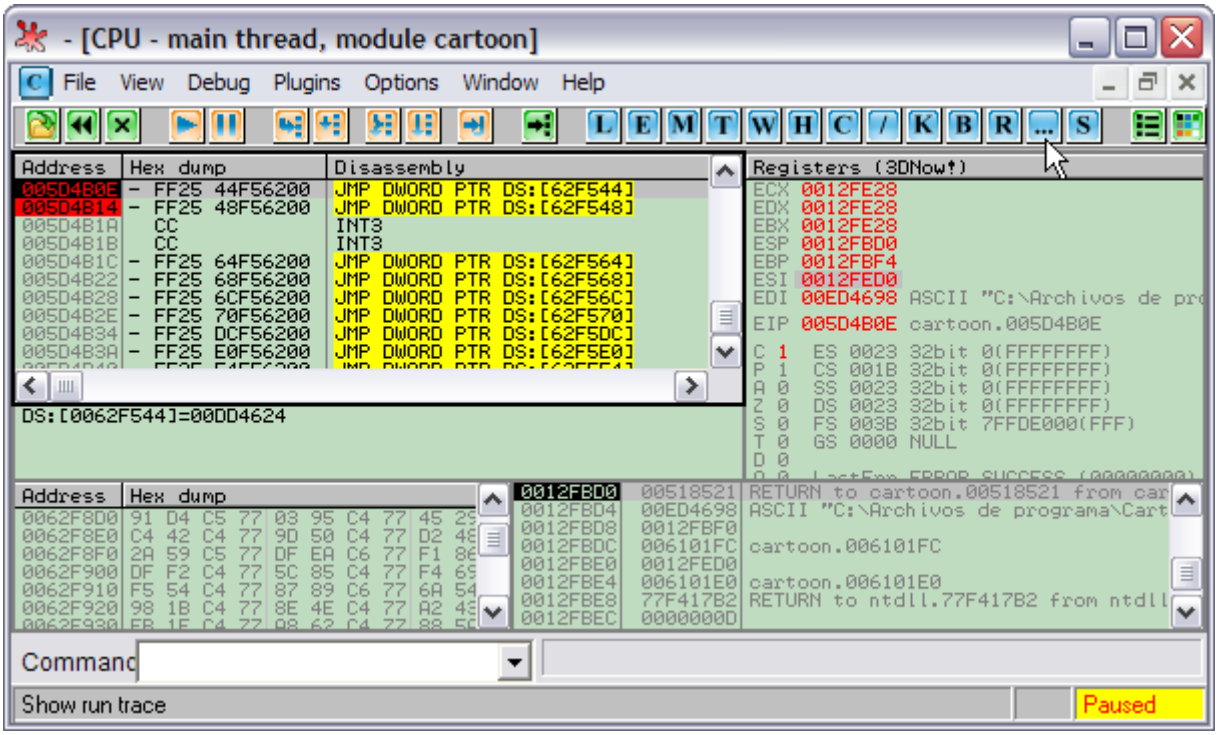
Estas son las que encontró para el primer grupo que va desde [62F540] hasta [62F548] incluidos ambos, cuando las tengamos pulsamos con el botón derecho y elejimos "Set breakpoint on every command" y pondrá un BP en cada una de ellas:



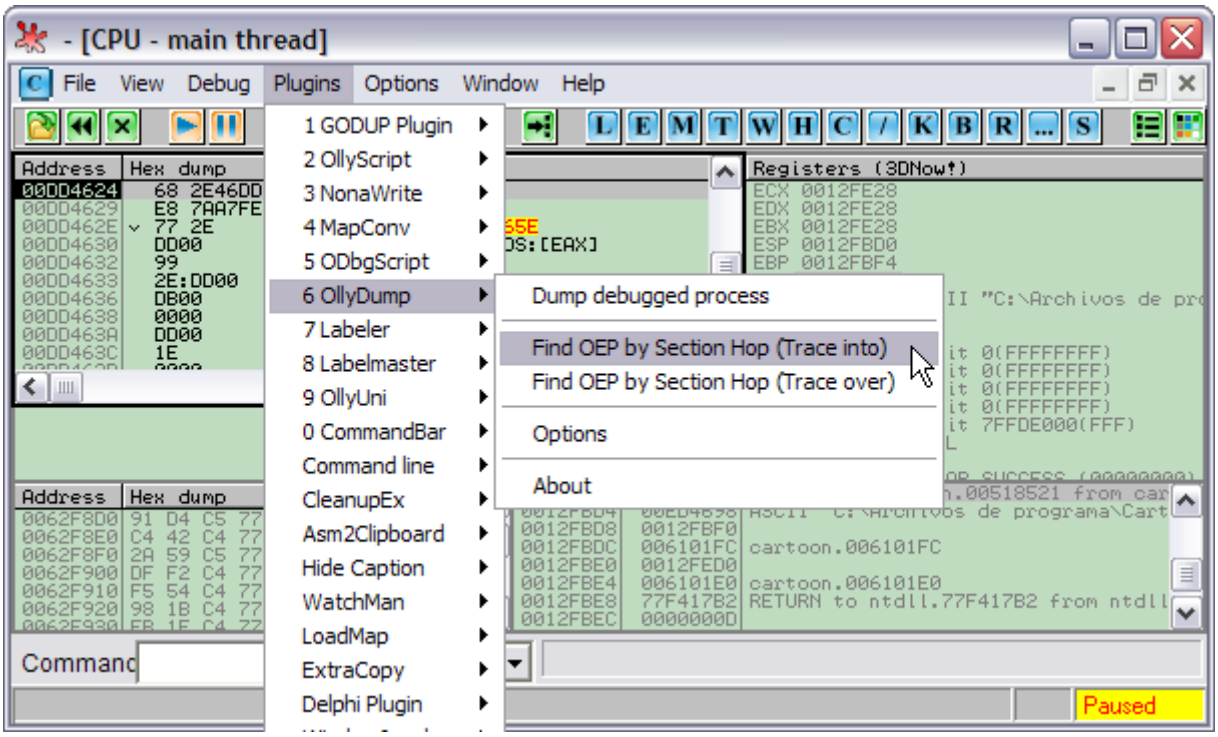
Ahí tenemos los BP's puestos y activados, jeje.. hacemos lo mismo con las dos librerías restantes (DLL's), al final cuando los tengamos todos dejamos correr el programa que teníamos parado en el OEP mientras hacíamos esto, pulsamos F9 y al momento Olly para, si miramos en la ventana de código veremos que paró a causa de uno de nuestros Breakpoints, justo en esta parte:

Address	Hex dump	Disassembly	Comment
005D4B0E	- FF25 44F56200	JMP DWORD PTR DS:[62F544]	
005D4B14	- FF25 48F56200	JMP DWORD PTR DS:[62F548]	
005D4B1A	CC	INT3	

Paró a causa del salto de :5D4B0E que va a la IAT a buscar la dirección de la api, leerá de [62F544] pero allí no está la dirección de la api sino una dirección que nos envía al código de ASProtect, en el original no habrá problema porque el packer estára cargado en memoria pero... en el dumpeado BOUUUUM petará por todos lados puesto que packer no habrá y estas direcciones de memoria no estarán preparadas. Lo que vamos a hacer es obtener la api a la que va después del salto que nos lleva al código del packer, podríamos trazarlo por completo y lo obtendríamos igual pero hay una opción en Olly mucho más cómoda.. el Tracer (Trazador) vamos a salvar en un archivo de texto todos los movimientos que haga por si luego intenta borrar sus pasos, cosa que no creo pero bueno, después lo pondremos a tracear. Para hacer un "Log" pulsamos en el botoncillo de los tres puntos suspensivos una única vez:

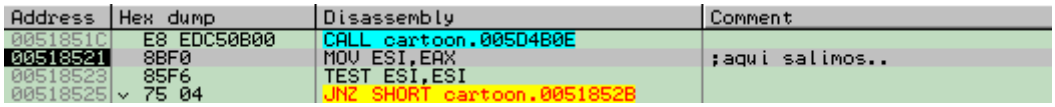


Se abrirá una pantallita donde deberemos decirle que nos lo guarde en el disco, esto lo haremos con el botón derecho y seleccionando "Log to file" y estará todo listo, ahora pongámoslo a tracear, con F7 para entramos en el código del packer y luego esto:



Y lo dejamos a su bola.., sabremos que vamos bien porque en la barra de estado veremos que está traceando gracias a este mensajito:

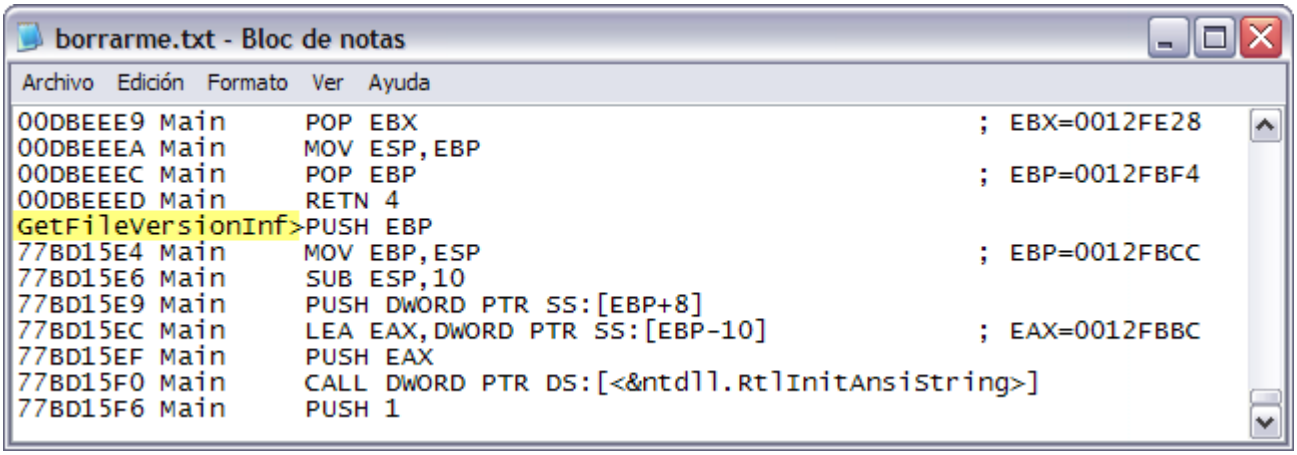
Al poco tiempo Olly para, veamos donde estamos:



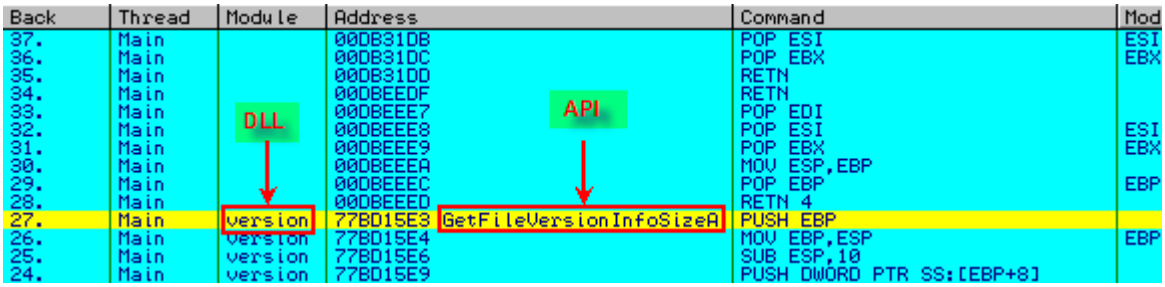
Address	Hex dump	Disassembly	Comment
0051851C	E8 EDC50B00	CALL cartoon.005D4B0E	
00518521	8BF0	MOV ESI,EAX	;aqui salimos..
00518523	85F6	TEST ESI,ESI	
00518525	75 04	JNZ SHORT cartoon.0051852B	

justo debajo del CALL que apunta al JMP a la IAT, si miramos la barra de estado vemos un "Paused" que indica que el programa vuelve a quedar a la espera de que "movamos ficha" de nuevo, pero antes de hacer nada vayamos a la ventanita Log y cerremos el archivo, con el botón derecho picamos y elejimos "Close log file" y ya tenemos el archivo preparado, cada uno le habrá puesto el nombre que haya querido, yo le puse "api_[62F544].txt" para diferenciarla del resto de entradas, pues son 18 ;-)

Lo que tenemos que hacer ahora es abrir el archivo de texto donde quedó guardada la información y mirar hasta enconrar una api:



Allí la vemos, jeje.. pero no se ve el nombre completo! no importa si miramos en la ventanita Log lo encontraremos sin problemas y bien completito ;-)

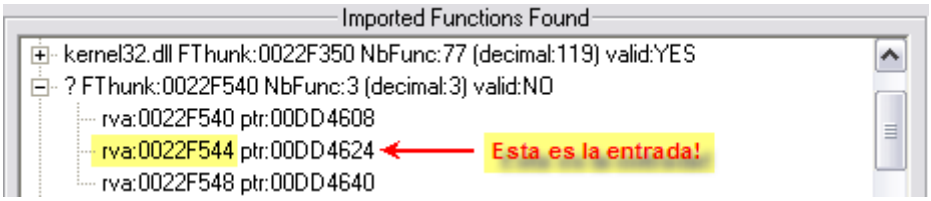


Back	Thread	Module	Address		Command	Mod
37.	Main		00DB31DB		POP ESI	ESI
36.	Main		00DB31DC		POP EBX	EBX
35.	Main		00DB31DD		RETN	
34.	Main		00DBEEDF		RETN	
33.	Main		00DBEEE7		POP EDI	
32.	Main		00DBEEE8		POP ESI	ESI
31.	Main		00DBEEE9		POP EBX	EBX
30.	Main		00DBEEEA		MOV ESP,EBP	
29.	Main		00DBEEEC		POP EBP	EBP
28.	Main		00DBEEED		RETN 4	
27.	Main	version	77BD15E3	GetFileVersionInfoSizeA	PUSH EBP	
26.	Main	version	77BD15E4		MOV EBP,ESP	EBP
25.	Main	version	77BD15E6		SUB ESP,10	
24.	Main	version	77BD15E9		PUSH DWORD PTR SS:[EBP+8]	

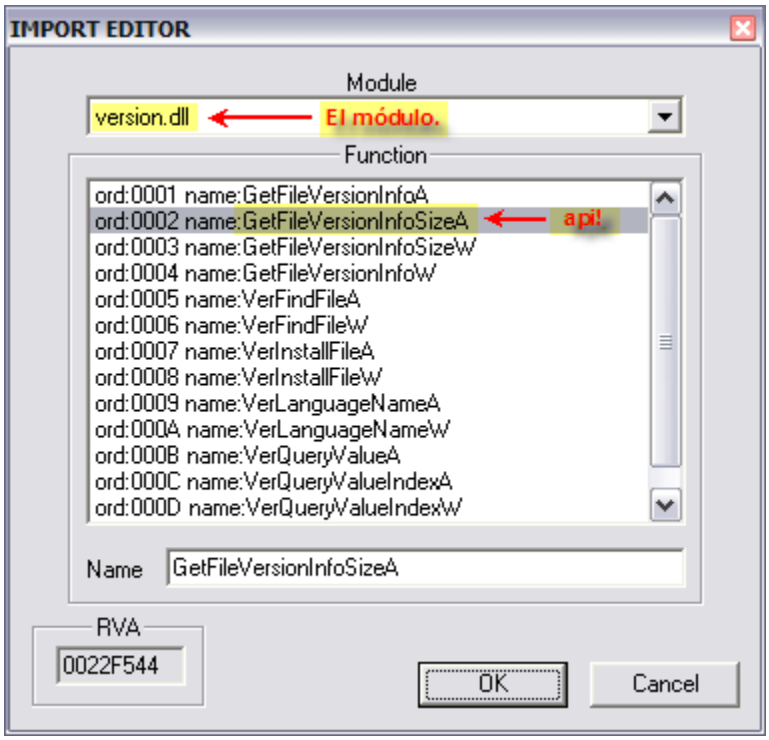
jeje.. ahí si vemos el nombre " GetFileVersionInfoSizeA" además de la DLL a la que pertenece "version.dll" lo único que nos queda hacer es ir al ImpRec y reparar la entrada cuyo RVA es 22F544 ¿porqué? muy sencillo el salto es este:

005D4B0E - FF25 44F56200 JMP DWORD PTR DS:[62F544]

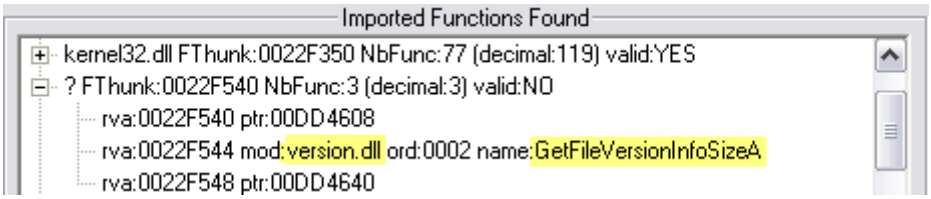
El salto apunta a la dirección 62F544, si a esta dirección le restamos el imagebase 400000 nos queda el RVA del mismo que es 22F544 que es el valor que encontraremos en el ImpREc referenciando a esta entrada:



Ahí vemos la entrada fácilmente reconocible por su 'rva' hacemos soble clic sobre ella y en la ventanita que aparece seleccionamos el módulo version.dll y la api correcta:



Cuando lo tengamos así pulsamos "OK" y la entrada queda reparada ;-)



Ahora sí tenemos la api completamente reparada. Este trabajo lo repetiremos para las 17 entradas diferentes, cada vez que encontremos una distinta quitamos el BP del JMP a la IAT para que no siga molestando, con esto además se consigue que cada vez que pare sea para una api distinta y que todavía no tenemos reparada.

Si seguimos este método veremos que reparamos todas las entradas excepto una ¡! hay una de ellas que por más que queramos no aparece por ningún lado, lo que vamos a hacer es cambiar de táctica, esta nos ha ido bastante bien pero no lo suficiente, hay veces en las que hay que mezclar varias técnicas diferentes y hoy es una de estas veces. Lo primero de todo será en saber cuál es la api a reparar:

Rva: 22F660 | Apunta a:DD46B0

Esta entrada tiene el siguiente JMP asociado:

005D4BD6 - FF25 60F66200 JMP DWORD PTR DS:[62F660]

lo que vamos a hacer es forzar al programa a ejecutar el salto, así llegaremos al código de ASProtect (el cual tracearemos claro xD), que de alguna manera debe saber qué api le hace falta al programa en este punto, es ahí cuando estaremos nosotros con los ojos bien abiertos para verla también, jajajajaja...

aunque tengamos el programa ya corriendo podemos hacerlo, primero pausamos Olly con F12, después hacemos un "Go" a la dirección del JMP :05D4BD6 y situamos el EIP allí con la opción "New origin here" acto seguido entramos en el código de ASProtect siguiendo el salto con F7, a partir de aquí es cuestión de traceo y muuuuucha calma, seguramente tenga que desencriptar el nombre de la api como hacía al inicio cuando rellenaba la iat, así que mucha paciencia ante todo...

Sin más preámbulos entremos al ataque ;-)

Recuperando 1 importación ;-)

Bueno chabales el packer está que se tambalea, parece un boxeador en el 12ª round y perdiendo contra las cuerdas, pero he de decir que todavía le queda un cartucho: una importación "perdida" no se sabe donde, jeje.. y que nosotros trataremos de encontrar, abróchense los cinturones que vamos palante ;-)

Como decía antes nos situamos encima del salto y situamos allí el EIP:

```
Address   Hex dump   Disassembly                                  Comment
005D4BD6  -          ---- ------- -- -- ------ -- -- -------- 0]
005D4BDC  -          Backup                          ▸ 4]       GDI32.Arc
005D4BE2  -          Copy                            ▸ 8]       GDI32.BitBlt
005D4BE8  -          Binary                          ▸ C]       GDI32.CombineRgn
005D4BEE  -          Assemble          Space           0]       GDI32.CopyEnhMetaFileA
005D4BF4  -          Label             :               4]       GDI32.CreateBitmap
005D4BFA  -          Comment           ;               8]       GDI32.CreateBrushIndirect
005D4C00  -          Breakpoint                      ▸ C]       GDI32.CreateCompatibleBitmap
005D4C06  -          Run trace                       ▸ 0]       GDI32.CreateCompatibleDC
005D4C0C  -                                            4]       GDI32.CreateDCA
005D4C12  -          Follow            Enter           8]       GDI32.CreateDIBSection
005D4C18  -          New origin here   Ctrl+Gray     C]       GDI32.CreateDIBitmap
005D4C1E  -          Go to                           ▸ 0]       GDI32.CreateFontIndirectA
005D4C24  -                                            4]       GDI32.CreateHalftonePalette
005D4C2A  -                                            8]       GDI32.CreateICA
005D4C30  -                                            C]       GDI32.CreatePalette
005D4C36  -                                            0]       GDI32.CreatePen
005D4C3C  -                                            4]       GDI32.CreatePenIndirect
005D4C42  -                                            8]       GDI32.CreatePolygonRgn
005D4C48  -                                            C]       GDI32.CreateRectRgn
005D4C4E  -                                            0]       GDI32.CreateSolidBrush
005D4C54  -                                            4]       GDI32.DeleteDC
```

Y quedará allí apuntando a la instrucción del salto, si ahoa pulsamos F7 vamos a la posición a la que apunta, en un principio debería ir directamente a una api, pero sabemos que ASProtect se ha metido por el medio, quitémoslo!

Al pulsar F7 caemos directamente en el código del packer, justo aquí:

```
Address   Hex dump   Disassembly                    Comment
00DD46B0    68 BA46DD00   PUSH 0DD46BA
00DD46B5    E8 EEA6FEFF   CALL 00DBEDA8
00DD46BA    2C 31         SUB AL,31
00DD46BC    DD00          FLD QWORD PTR DS:[EAX]
00DD46BE    6E            OUTS DX,BYTE PTR ES:[EDI]      I/O command
00DD46BF    31DD          XOR EBP,EBX
00DD46C1    00DB          ADD BL,BL
00DD46C3    0000          ADD BYTE PTR DS:[EAX],AL
00DD46C5    00FF          ADD BH,BH
00DD46C7    FF1E          CALL FAR FWORD PTR DS:[ESI]    Far call
00DD46C9    0000          ADD BYTE PTR DS:[EAX],AL
```

Caemos en el PUSH inicial, desde aquí iremos traceando pero siempre con F7, este es capaz de tirarnos para afuera si intentamos pasar los CALL con F8, no lo hge comprobado pero no me fio ni un pelo, al poco de comenzar a tracear se empiezan a ver cosas interesantes, lo primero que nos llama la atención es esto:

```
Address   Hex dump   Disassembly                    Comment
00DBEDF8    8BC3          MOV EAX,EBX
00DBEDFA    8B50 04       MOV EDX,DWORD PTR DS:[EAX+4]
00DBEDFD    8A12          MOV DL,BYTE PTR DS:[EDX]       ;toma el Id. de la api
00DBEDFF    3A15 8627DC00 CMP DL,BYTE PTR DS:[DC2786]    ;lo compara..
00DBEE05  v 74 45         JE SHORT 00DBEE4C
00DBEE07    8B50 04       MOV EDX,DWORD PTR DS:[EAX+4]
```

En :0DBEDFD pone el Identificador de api en DL, si en este momento hacemos un "Follow in dump->Memory Address" veremos la zona de memoria a la que hace referencia:

```
Address   Hex dump                                        ASCII
00DD316E  4A 0A 7F 83 F7 0F 08 EE BC 43 29 F9 5A F4 F7 22  J.⌂â▪*■¯ª C)¨Z¶▪"
00DD317E  00 01 67 64 69 33 32 2E 64 6C 6C 00 4A 04 FA 77  .0gdi32.dll.J♦·w
00DD318E  5A AC 4A 07 F9 6C 4D EE D4 68 EE 4A 0B F8 6A 54  Z¼J•·lM¯ÊhÔJ♂°jT
00DD319E  CE D1 72 8B 12 B8 DF 28 4A 11 F8 6A 49 D5 FD 72  ╬Ñrï↨↑▀(J◄°jI'²r
```

Está leyendo el valor "4A" que es el citado Identificador, después viene "0A" que es el tamaño del nombre de la api, y luego el nombre de ésta pero encriptado, jeje.. casi lo tenemos, sólo falta que lo desencripte, así que sigamos...

Vamos traceando por el código de ASProtect y llegamos a una zona que ya nos es familiar, esto está justo después de la zona anteriormente vista donde tomaba el valor "4A" correspondiente al id.api, justo después llega un JE, que en caso de no producirse llegamos a la zona que ahora veremos:

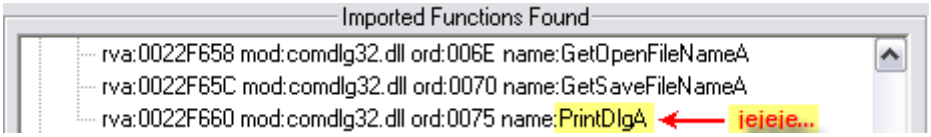
```
Address   Hex dump     Disassembly                      Comment
00DBEDFA  8B50 04      MOV EDX,DWORD PTR DS:[EAX+4]
00DBEDFD  8A12         MOV DL,BYTE PTR DS:[EDX]         ;toma el Id. de la api
00DBEDFF  3A15 8627DC00 CMP DL,BYTE PTR DS:[DC2786]     ;lo compara..
00DBEE05  74 45        JE SHORT 00DBEE4C
00DBEE07  8B50 04      MOV EDX,DWORD PTR DS:[EAX+4]
00DBEE0A  8BCA         MOV ECX,EDX
00DBEE0C  41           INC ECX
00DBEE0D  8A19         MOV BL,BYTE PTR DS:[ECX]
00DBEE0F  83C2 02      ADD EDX,2
00DBEE12  8955 FC      MOV DWORD PTR SS:[EBP-4],EDX
00DBEE15  8BF3         MOV ESI,EBX
00DBEE17  81E6 FF000000 AND ESI,0FF
00DBEE1D  8BCE         MOV ECX,ESI
00DBEE1F  8D85 F8FEFFFF LEA EAX,DWORD PTR SS:[EBP-108]
00DBEE25  8B55 FC      MOV EDX,DWORD PTR SS:[EBP-4]
00DBEE28  E8 1F56FFFF  CALL 00DB444C                    ;copia api encriptada!!
00DBEE2D  6A 0A        PUSH 0A
00DBEE2F  B9 7827DC00  MOV ECX,0DC2778
00DBEE34  8BD6         MOV EDX,ESI
00DBEE36  8D85 F8FEFFFF LEA EAX,DWORD PTR SS:[EBP-108]
00DBEE3C  E8 B3D2FFFF  CALL 00DBC0F4                    ;la desencripta!!
00DBEE41  8D85 F8FEFFFF LEA EAX,DWORD PTR SS:[EBP-108]
00DBEE47  8945 FC      MOV DWORD PTR SS:[EBP-4],EAX
00DBEE4A  EB 09        JMP SHORT 00DBEE55
00DBEE4C  8B40 04      MOV EAX,DWORD PTR DS:[EAX+4]
00DBEE4F  40           INC EAX
```

justo en esos dos CALL seguidos es donde copia y desencripta la api, para verlo con más detalle nos ponemos la ventana dump a la vista donde sabemos que la copiará [12FDC4] esto lo sabemos gracias a los registros, tras pasar el segundo CALL.. tachaaaaaan...

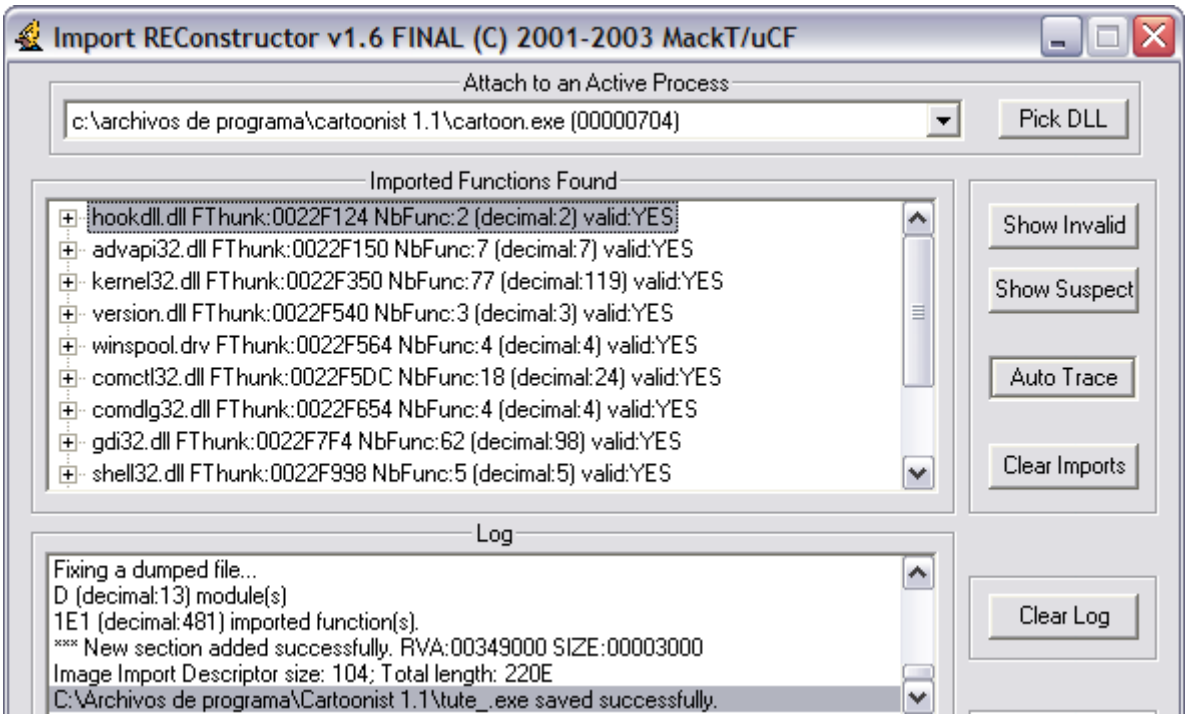
```
Address   Hex dump                                         ASCII
0012FDC4  50 72 69 6E 74 44 6C 67 41 00 00 00 00 00 00 00  PrintDlgA.......
0012FDD4  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0012FDE4  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0012FDF4  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

jejejeje.. ahí tenemos la API, es PrintDlgA una función que no encuentro en el programa (la de imprimir) es por esto que no conseguíamos hacer saltar al Olly en este JMP ¡No utiliza la API!, o al menos yo no encuentro la función de imprimir. Pero más importante de este método es que no necesitamos que el programa utilice la funció para conocer su nombre ;-)

Una vez lo tenemos vamos al Import y rellenamos la entrada como Dios manda:



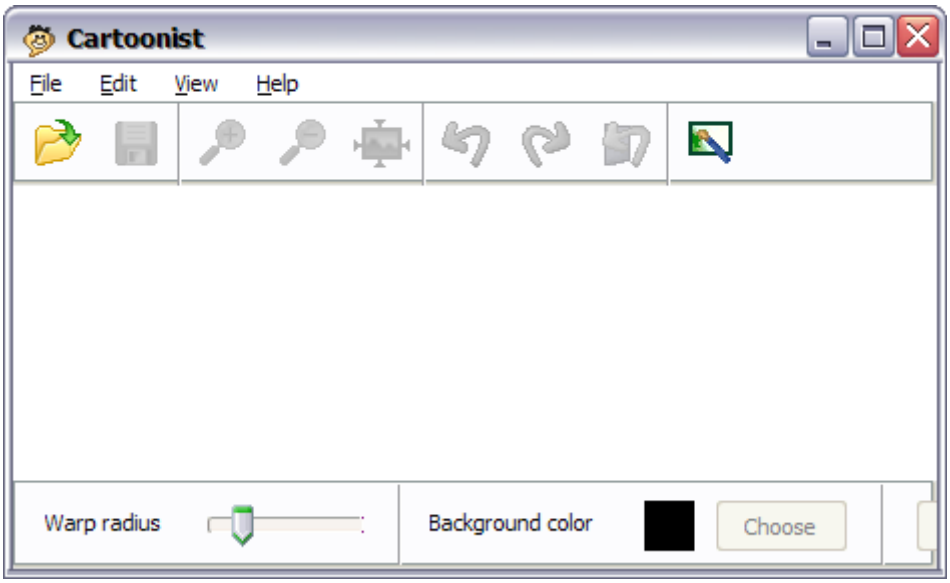
Y ya está, tenemos la tabla totalmente resuelta, es el momento de reparar el dumpeado a ver si funciona, pulsamos "Fix Dump" y elegimos tute.exe o como lo hayamos llamado (al dumpeado me refiero) al momento nos informa que todo fue bien ;-)



S

alvó el archivo "tute_.exe" que en principio no nos debería dar problemas, llegó el momento de ejecutarlo a ver si funciona...

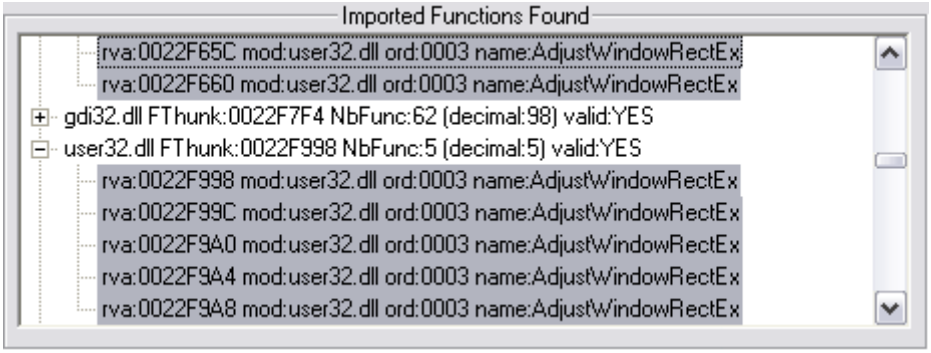
Le hacemos doble clic y...



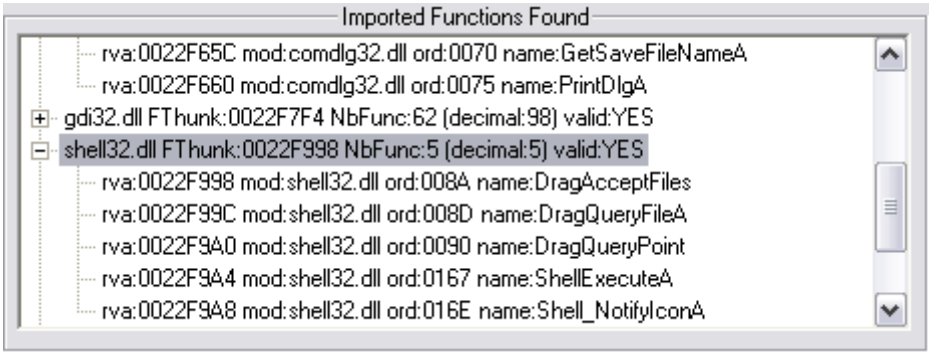
funcionaaaaaaaaaaaaaaa...!!

Pues bien hasta aquí hemos llegado con el desempakado, en una segunda parte veremos cómo quitar las limitaciones del programa, es esta hemos visto cómo desempakar un ASProtect 1.1c que es de las versiones más sencillas del packer aunque la redirección de la IAT es algo distinta de versiones más nuevas, en las que con sólo nopear el mágico queda un máximo con 11 entradas malas. En fin espero que os haya gustado, por mi parte ha sido un placer, hasta la próxima!!

COMENTARIO: Como comentario adicional quería mostrar cómo el Import Reconstructor a veces no repara del todo bien la IAT, pondré un ejemplo de este mismo programa. LLegaremos al OEP con el método conocido de las excepciones, abrimos el ImpRec y nos quedan 347 entradas no resueltas, si picamos en "Show Invalid" y después le pasamos el "Trace Level1 (Disasm)" nos dirá alegremente que sólo quedaron 2 entradas por resolver, pero esto no es del todo cierto, echemos un vistazo:



Muchas de ellas la resolvió como "AdjustWindowRectExA" cuando en realidad se trata de otras apis, veamos la iat correctamente resuelta para como deberían quedar:



Como véis el resultado no es el mismo, no sé si esto es a causa de un bug del programa o de donde viene, pero lo cierto es que nos quiere hacer hacer ver que estas entradas etán reparadas cuando en realidad siguen todas sin resolver, puesto que les ha asignado una api que no corresponde en ningún caso, es más, las repara como si fueran del user32.dll cuando en realidad son de la commdlg32.dll y del shell32.dll. En fin creo que no estaba de más mostrarlo.

Y ahora sí termina el tute de hoy, ya era hora, jeje.. para cualquier duda, sugerencia, pregunta o lo que sea me escribís como siempre a: syxe00@yahoo.es

Un saludo y un abrazo a TODO CRACKSLATINOS!!

SyXe'05